

OmniFocus 3.13.1 (v151.27.0)

[Alert](#) • [Application](#) • [ApplyResult](#) • [Array](#) • [FolderArray](#) • [ProjectArray](#) • [SectionArray](#) • [Library](#) • [TagArray](#) • [Tags](#) • [TaskArray](#) • [Inbox](#) • [Audio](#) • [Audio.Alert](#) • [Calendar](#) • [Color](#) • [ColorSpace](#) • [Console](#) • [Credentials](#) • [Crypto](#) • [Crypto.SHA256](#) • [Crypto.SHA384](#) • [Crypto.SHA512](#) • [Data](#) • [Database](#) • [DatabaseObject](#) • [DatedObject](#) • [ActiveObject](#) • [Folder](#) • [Tag](#) • [Task](#) • [Perspective.Custom](#) • [Task.Notification](#) • [Project](#) • [DateComponents](#) • [DateRange](#) • [Decimal](#) • [Device](#) • [DeviceType](#) • [Document](#) • [DatabaseDocument](#) • [Email](#) • [Error](#) • [FilePicker](#) • [FileSaver](#) • [FileWrapper](#) • [FileWrapper.ReadingOptions](#) • [FileWrapper.Type](#) • [FileWrapper.WritingOptions](#) • [Folder.ChildInsertionLocation](#) • [Folder.Status](#) • [ForecastDay](#) • [ForecastDay.Kind](#) • [ForecastDay.Status](#) • [Form](#) • [Form.Field](#) • [Form.Field.Checkbox](#) • [Form.Field.Date](#) • [Form.Field.MultipleOptions](#) • [Form.Field.Option](#) • [Form.Field.Password](#) • [Form.Field.String](#) • [Formatter](#) • [Formatter.Date](#) • [Formatter.Decimal](#) • [Formatter.Duration](#) • [Formatter.Date.Style](#) • [Function](#) • [Image](#) • [LigatureStyle](#) • [Locale](#) • [MenuItem](#) • [NamedStyle.List](#) • [NamedStylePosition](#) • [Notification](#) • [ObjectIdentifier](#) • [Pasteboard](#) • [Pasteboard.Item](#) • [Perspective](#) • [Perspective.BuiltIn](#) • [PlugIn](#) • [PlugIn.Action](#) • [PlugIn.Handler](#) • [PlugIn.Library](#) • [Preferences](#) • [Project.ReviewInterval](#) • [Project.Status](#) • [Promise](#) • [Selection](#) • [Settings](#) • [SharePanel](#) • [Speech](#) • [Speech.Boundary](#) • [Speech.Synthesizer](#) • [Speech.Utterance](#) • [Speech.Voice](#) • [Speech.Voice.Gender](#) • [StringEncoding](#) • [Style](#) • [NamedStyle](#) • [Style.Attribute](#) • [Tag.ChildInsertionLocation](#) • [Tag.Status](#) • [Task.ChildInsertionLocation](#) • [Task.Notification.Kind](#) • [Task.RepetitionMethod](#) • [Task.RepetitionRule](#) • [Task.Status](#) • [Text](#) • [Text.FindOption](#) • [Text.Position](#) • [Text.Range](#) • [TextAlignment](#) • [TextComponent](#) • [TimeZone](#) • [Timer](#) • [ToolbarItem](#) • [Tree](#) • [ContentTree](#) • [SidebarTree](#) • [TreeNode](#) • [TypeIdentifier](#) • [URL](#) • [URL.Access](#) • [URL.Bookmark](#) • [URL.Components](#) • [URL.FetchRequest](#) • [URL.FetchResponse](#) • [URL.QueryItem](#) • [UnderlineAffinity](#) • [UnderlinePattern](#) • [UnderlineStyle](#) • [Version](#) • [Window](#) • [DocumentWindow](#) • [XML](#) • [XML.Document](#) • [XML.Document.Configuration](#) • [XML.Element](#) • [XML.WhitespaceBehavior](#) • [XML.WhitespaceBehavior.Type](#)

Alert

An alert interface for displaying information to the user, blocking further interaction until the alert is dismissed.

Constructors

`new Alert(title: String, message: String) → Alert`

Create a new alert panel with the given title and text contents.

Instance Functions

`function show(callback: Function(option: Number) or null) → Promise of Number`

Displays the alert. If no options have yet been added, a default “OK” option is added. Once the user selects an option, the alert is dismissed. If a callback function was supplied, it is invoked with the zero-based index of the selected option as its argument. A Promise is returned as well, which may also be used to collect the result of the Alert.

`function addOption(string: String)`

Adds an option button to the alert.

Application

Instance Functions

```
function openDocument(from: Document or null, url: URL, completed: Function(documentOrError: Document or Error, alreadyOpen: Boolean))
```

Attempts to open the specified document and return a reference to it asynchronously. If the document is already open, the reference is passed along. Note that due to platform sandboxing restrictions, opening the document may fail if the application doesn't have currently permission to access the given URL. The document, if any, that is associated with the calling script can be passed along to help grant permission to open the new document. The passed in function will be passed two argument. The first will be either either the Document or an Error. On success, the second argument is a Boolean specifying whether the document was already open.

Instance Properties

```
var buildVersion → Version read-only
```

The internal build version number for the app. See also `userVersion`.

```
var commandKeyDown → Boolean read-only
```

Whether the Command key is currently down.

```
var controlKeyDown → Boolean read-only
```

Whether the Control key is currently down.

```
var name → String read-only
```

Application name.

var `optionKeyDown` → Boolean *read-only*

Whether the Option key is currently down.

var `platformName` → String *read-only*

Returns a string describing the current platform, currently "iOS" or "macOS".

var `shiftKeyDown` → Boolean *read-only*

Whether the Shift key is currently down.

var `userVersion` → [Version](#) *read-only*

The user-visible version number for the app. See also `buildVersion`.

var `version` → String *read-only*

Deprecated: Recommend using either `userVersion` or `buildVersion`.

For backwards compatibility with existing scripts, this returns the same result as `buildVersion.versionString`. We recommend using either the user-visible `userVersion` or the internal `buildVersion` instead, which are more clear about which version they're returning and provide their results as `Version` objects which can be semantically compared with other `Version` objects.

ApplyResult

Class Properties

var `SkipChildren` → [ApplyResult](#) *read-only*

The descendants of the current item are skipped.

var `SkipPeers` → [ApplyResult](#) *read-only*

The unvisited peers of the current item are skipped.

var `Stop` → [ApplyResult](#) *read-only*

The call to `apply` terminates with no further items being visited.

var `all` → Array of [ApplyResult](#) *read-only*

Array

The built-in JavaScript [Array](#) constructor.

FolderArray : [Array](#)

An Array containing `Folder` objects.

Instance Functions

function `byName`(name: String) → [Folder](#) or null

Returns the first Fo`lder` contained directly in this array with the given name.

ProjectArray : [Array](#).

An Array containing Project objects.

Instance Functions

function `byName`(name: String) → [Project](#) or null

Returns the first Project contained directly in this array with the given name.

SectionArray : [Array](#).

An Array containing Project and Fo`lder` objects.

Instance Functions

function `byName`(name: String) → [Project](#) or [Folder](#) or null

Returns the first Project or Fo`lder` contained directly in this array with the given name.

Library : [SectionArray](#).

An Array of folders and projects at the top level of the database. (This can be referenced via the top-level global variable `library`.)

Instance Functions

function `apply`(function: [Function](#)) → [ApplyResult](#) or null

Calls the given function for each Folder and Project in the Library and recursively into any child folders. Note that the tasks in projects are not descended into.

Instance Properties

var `beginning` → [Folder.ChildInsertionLocation](#) *read-only*

Returns a location referring to the beginning of the top-level projects and folders in the database. (Reference this using `library.beginning`.)

var `ending` → [Folder.ChildInsertionLocation](#) *read-only*

Returns a location referring to the ending of the top-level projects and folders in the database. (Reference this using `library.ending`.)

TagArray : [Array](#)

An Array containing Tag objects.

Instance Functions

function `byName`(name: String) → [Tag](#) or null

Returns the first Tag contained directly in this array with the given name.

Tags : [TagArray](#)

An Array of tags at the top level of the database.

Instance Functions

function `apply`(function: [Function](#)) → [ApplyResult](#) or null

Calls the given function for each Tag in the Library and recursively into any child tags.

Instance Properties

var `beginning` → [Tag.ChildInsertionLocation](#) *read-only*

Returns a location referring to the beginning of the top-level tags in the database.

var `ending` → [Tag.ChildInsertionLocation](#) *read-only*

Returns a location referring to the ending of the top-level tags in the database.

TaskArray : [Array](#)

An Array containing Task objects.

Instance Functions

function `byName`(name: String) → [Task](#) or null

Returns the first Task contained directly in this array with the given name.

Inbox : [TaskArray](#)

An Array of tasks that are in the inbox.

Instance Functions

function `apply`(function: [Function](#)) → [ApplyResult](#) or null

Calls the given function for each Task in the Inbox and recursively into any child tasks.

Instance Properties

var `beginning` → [Task.ChildInsertionLocation](#) *read-only*

A location that can be used when adding, duplicating, or moving tasks.

var `ending` → [Task.ChildInsertionLocation](#) *read-only*

A location that can be used when adding, duplicating, or moving tasks.

Audio

Class Functions

function `playAlert(alert: Audio.Alert or null, completed: Function() or null)`

Play the specified `Audio.Alert`. On macOS, if no alert is specified, the user's default alert sound will be played. On iOS, there is no default alert sound and nothing will be done without specifying an alert.

`Audio.Alert`

Constructors

`new Audio.Alert(url: URL) → Audio.Alert`

`Calendar`

Class Properties

`var buddhist → Calendar read-only`

`var chinese → Calendar read-only`

`var coptic → Calendar read-only`

`var current → Calendar read-only`

The user's preferred calendar

`var ethiopicAmeteAlem → Calendar read-only`

var `ethiopicAmeteMihret` → [Calendar](#) *read-only*

var `gregorian` → [Calendar](#) *read-only*

The Gregorian calendar.

var `hebrew` → [Calendar](#) *read-only*

var `indian` → [Calendar](#) *read-only*

var `islamic` → [Calendar](#) *read-only*

var `islamicCivil` → [Calendar](#) *read-only*

var `islamicTabular` → [Calendar](#) *read-only*

var `islamicUmmAlQura` → [Calendar](#) *read-only*

var `iso8601` → [Calendar](#) *read-only*

var `japanese` → [Calendar](#) *read-only*

var `persian` → [Calendar](#) *read-only*

var `republicOfChina` → [Calendar](#) *read-only*

Instance Functions

function `dateByAddingDateComponents`(date: Date,
components: [DateComponents](#)) → Date or null

Returns a new Date by adding the given DateComponents, or null if no date could be calculated.

function `dateFromDateComponents`(components: [DateComponents](#)) → Date **or** null

Returns a new Date from the given DateComponents, or null if no date could be calculated.

function `dateComponentsFromDate`(date: Date) → [DateComponents](#)

Returns a new DateComponents for the given Date.

function `dateComponentsBetweenDates`(start: Date, end: Date) → [DateComponents](#)

Returns the difference from the start Date to the end Date as a DateComponents.

function `startOfDay`(date: Date) → Date

Returns a Date for the first moment of the day containing the given Date according to this Calendar.

Instance Properties

var `identifier` → String ***read-only***

The ISO identifier for the calendar.

var `locale` → [Locale](#) **or** null ***read-only***

The locale of the calendar.

var `timeZone` → [TimeZone](#) *read-only*

The time zone of the calendar.

Color

Class Functions

function `RGB`(r: Number, g: Number, b: Number, a: Number or null) → [Color](#)

Makes a new color in the RGB colorspace, with the given components. If the alpha component is not given, 1.0 is used.

function `HSB`(h: Number, s: Number, b: Number, a: Number or null) → [Color](#)

Makes a new color in the HSB colorspace, with the given components. If the alpha component is not given, 1.0 is used.

function `White`(w: Number, a: Number or null) → [Color](#)

Makes a new color in the White colorspace, with the given components. If the alpha component is not given, 1.0 is used.

Class Properties

var **black** → Color *read-only*

A color in the White colorspace with white component of 0.0.

var **blue** → Color *read-only*

A color in the RGB colorspace with components (0, 0, 1, 1).

var **brown** → Color *read-only*

A color in the RGB colorspace with components (0.6, 0.4, 0.2, 1).

var **clear** → Color *read-only*

A color in the White colorspace with white component of 0.0 and alpha of 0.0 (“transparent black”).

var **cyan** → Color *read-only*

A color in the RGB colorspace with components (0, 1, 1, 1).

var **darkGray** → Color *read-only*

A color in the White colorspace with white component of 0.333.

var **gray** → Color *read-only*

A color in the White colorspace with white component of 0.5.

var **green** → Color *read-only*

A color in the RGB colorspace with components (0, 1, 0, 1).

var `lightGray` → Color *read-only*

A color in the White colorspace with white component of 0.667.

var `magenta` → Color *read-only*

A color in the RGB colorspace with components (1, 0, 1, 1).

var `orange` → Color *read-only*

A color in the RGB colorspace with components (1, 0.5, 0, 1).

var `purple` → Color *read-only*

A color in the RGB colorspace with components (1, 0, 1, 1).

var `red` → Color *read-only*

A color in the RGB colorspace with components (1, 0, 0, 1).

var `white` → Color *read-only*

A color in the White colorspace with white component of 1.0.

var `yellow` → Color *read-only*

A color in the RGB colorspace with components (1, 1, 0, 1).

Instance Functions

function `blend`(otherColor: [Color](#), fraction: Number) → [Color](#) or null

Returns a new color that is a linear combination of the receiver and fraction of the other color (so, a fraction of 1.0 would just return the otherColor. If the colors cannot be blended (for example, if they cannot be converted to the same colorspace), then null is returned.

Instance Properties

var `alpha` → Number *read-only*

Returns the alpha component of the color.

var `blue` → Number *read-only*

Returns the blue component of the color, after converting to a RGB colorspace.

var `brightness` → Number *read-only*

Returns the brightness component of the color, after converting to a HSB colorspace.

var `colorSpace` → [ColorSpace](#) *read-only*

Returns the colorspace of the instance.

var `green` → Number *read-only*

Returns the green component of the color, after converting to a RGB colorspace.

var **hue** → Number *read-only*

Returns the hue component of the color, after converting to a HSB colorspace.

var **red** → Number *read-only*

Returns the red component of the color, after converting to a RGB colorspace.

var **saturation** → Number *read-only*

Returns the saturation component of the color, after converting to a HSB colorspace.

var **white** → Number *read-only*

Returns the white component of the color, after converting to a White colorspace.

ColorSpace

Class Properties

var **CMYK** → [ColorSpace](#) *read-only*

A colorspace with cyan, magenta, yellow, black, and alpha components.

var **HSB** → [ColorSpace](#) *read-only*

A colorspace with hue, saturation, and value (or brightness) components.

var **Named** → ColorSpace *read-only*

A space for named colors, like system defined colors, or specific color palette spaces.

var **Pattern** → ColorSpace *read-only*

A colorspace that wraps a pattern image.

var **RGB** → ColorSpace *read-only*

The sRGB colorspace with red, green, blue, and alpha components.

var **White** → ColorSpace *read-only*

A colorspace with white and alpha components.

var **all** → Array of ColorSpace *read-only*

Console

The Console allows scripts to log debugging, warning, or error information where it can be viewed in the system console or in the console output area. A single instance of Console is available to scripts as the console global variable.

Instance Functions

function `log`(message: [Object](#), additional: Array of [Object](#) or null)

Appends a line to the application console formed by concatenating the given message (after converting it to a `String`), any additional arguments separated by spaces, and finally a newline.

function `error`(message: [Object](#), additional: Array of [Object](#) or null)

function `info`(message: [Object](#), additional: Array of [Object](#) or null)

function `warn`(message: [Object](#), additional: Array of [Object](#) or null)

Just calls `Console.log`, currently.

function `clear`()

Clears the console in the user-visible window.

Credentials

The `Credentials` class allows storage of private username and password pairs, `URL.Bookmark` instances, and possibly other sensitive information in the future. Instances are tied to a single plug-in and single application, and may only be created in plug-ins when they are being loaded.

For example, when a `PlugIn.Action` is being created, you could use the following pattern:

```
(() => {  
    let credentials = new Credentials();  
  
    var action = new PlugIn.Action(function(selection) {  
        // ... use the captured credentials ...  
    });  
  
    return action;  
})();
```

Attempts to create `Credentials` instances elsewhere will throw an error. Care should be taken to store instances in anonymous closures as above, and not pass them to or store them on other objects.

Credentials are keyed off a service identifier, which your plug-in can define however it likes.

Constructors

new `Credentials()` → [`Credentials`](#)

Creates a new `Credentials` instance for the currently loading plug-in. Throws an error if called outside of plug-in loading.

Instance Functions

function `read(service: String)` → **Object or null**

Looks up the current credentials for a given service identifier. If credentials have previously been stored, an object will be returned containing "user" and "password" properties.

```
function write(service: String, username: String,  
password: String)
```

Creates or updates an existing credential, storing the username and password for this service securely in the Keychain.

```
function remove(service: String)
```

Deletes any currently stored credentials for the specified service, either a username and password or a URL bookmark.

```
function readBookmark(service: String) → URL.Bookmark or  
null
```

Reads the entry for the given service identifier and attempts to return it as a `URL.Bookmark`, or `null` if no such entry exists.

```
function writeBookmark(service: String, bookmark:  
URL.Bookmark)
```

Stores the `URL.Bookmark` persistently for later access.

Crypto

Crypto provides access to some of Apple's [CryptoKit](#)

Class Functions

```
function randomData(length: Number) → Data
```

Copy length bytes of cryptographically secure random data.

Crypto.SHA256

The SHA-256 hash function.

Constructors

`new Crypto.SHA256()` → [Crypto.SHA256](#)

Create a new SHA-256 digest.

Instance Functions

`function update(data: Data)`

Incrementally update the digest with the given data.

`function finalize() → Data`

Finalize any remaining digest process and return the result of the hash function.

Crypto.SHA384

The SHA-384 hash function.

Constructors

`new Crypto.SHA384()` → [Crypto.SHA384](#)

Create a new SHA-384 digest.

Instance Functions

`function update(data: Data)`

Incrementally update the digest with the given data.

`function finalize() → Data`

Finalize any remaining digest process and return the result of the hash function.

Crypto.SHA512

The SHA-512 hash function.

Constructors

`new Crypto.SHA512()` → [Crypto.SHA512](#)

Create a new SHA-512 digest.

Instance Functions

`function update(data: Data)`

Incrementally update the digest with the given data.

function `finalize()` → [Data](#)

Finalize any remaining digest process and return the result of the hash function.

Data

A generic bag of bytes. Mainly useful to be interpreted / converted to some other type.

Class Functions

function `fromString(string: String, encoding: StringEncoding or null)` → [Data](#)

Convert the string to a Data using the given encoding, or UTF8 if none is specified.

function `fromBase64(string: String)` → [Data](#)

Instance Functions

function `toString(encoding: StringEncoding or null)` → String

Convert to a String, assuming that this Data using the specified encoding, or UTF8 if none is given.

function `toBase64()` → String

Convert to a Base-64 encoded string.

Instance Properties

var `length` → Number *read-only*

Number of bytes in this data.

var `toObject` → [Object](#) or null *read-only*

Database

Instance Functions

function `tagNamed`(name: String) → [Tag](#) or null

Returns the first top-level Tag with the given name, or null.

function `folderNamed`(name: String) → [Folder](#) or null

Returns the first top-level Folder with the given name, or null.

function `projectNamed`(name: String) → [Project](#) or null

Returns the first top-level Project with the given name, or null.

function `projectsMatching`(search: String) → Array of [Project](#)

Returns each existing Project that Smart Matches the given search. The result will be in the same order and have the same projects as would be found when searching this string in the Quick Open window.

function `foldersMatching`(search: String) → Array of [`Folder`](#)

Returns each existing Folder that Smart Matches the given search. The result will be in the same order and have the same folders as would be found when searching this string in the Quick Open window.

function `tagsMatching`(search: String) → Array of [`Tag`](#)

Returns each existing Tag that Smart Matches the search. The result will be in the same order and have the same tags as would be found when searching this string in the Quick Open window.

function `taskNamed`(name: String) → [`Task`](#) or null

Returns the first top-level Task in the inbox with the given name, or null.

function `save`()

Saves any unsaved changes to disk. If sync is enabled and there were unsaved changes, this also triggers a sync request.

function `moveTasks`(tasks: Array of [`Task`](#), position: [`Project`](#), [`Task`](#), or [`Task.ChildInsertionLocation`](#))

Moves tasks to a different location.

function `duplicateTasks`(tasks: Array of [Task](#),
position: [Project](#), [Task](#), or
[Task.ChildInsertionLocation](#)) → [TaskArray](#)

Makes copies of the tasks and returns the new copies. The order of the inputs is not considered and the copies are returned in library order of the originals.

function `convertTasksToProjects`(tasks: Array of [Task](#),
position: [Folder](#) or [Folder.ChildInsertionLocation](#)) →
Array of [Project](#)

Converts tasks to new projects at the specified location.

For example, to convert each top-level inbox item into a new project at the end of your library and capture the resulting projects:

```
const newProjects = convertTasksToProjects(inbox, library.ending);
```

function `moveSections`(sections: Array of [Project](#) or
[Folder](#), position: [Folder](#) or
[Folder.ChildInsertionLocation](#))

Moves sections to a different location.

function `duplicateSections`(sections: Array of [Project](#)
or [Folder](#), position: [Folder](#) or
[Folder.ChildInsertionLocation](#)) → [SectionArray](#)

Makes copies of the sections and returns the new copies. The order of the inputs is not considered and the copies are returned in library order of the originals.

function `moveTags`(tags: Array of [Tag](#), position: [Tag](#) or [Tag.ChildInsertionLocation](#))

Moves tags to a different location.

function `duplicateTags`(tags: Array of [Tag](#), position: [Tag](#) or [Tag.ChildInsertionLocation](#)) → [TagArray](#)

Makes copies of the tags and returns the new copies. The order of the inputs is not considered and the copies are returned in library order of the originals.

function `cleanUp`()

Processes inbox items that have the required information to move into their proposed containers, performs any delayed filtering, and deletes empty items.

function `undo`()

Undoes the last undoable action, or throws an error if there are no undoable actions.

function `redo`()

Redoes the next redoable action, or throws an error if there are no redoable actions.

function `deleteObject`(object: [DatabaseObject](#))

Removes the object from the Database.

```
function copyTasksToPasteboard(tasks: Array of Task,  
pasteboard: Pasteboard)
```

Copies the given tasks to the pasteboard in a variety of formats.

```
function canPasteTasks(pasteboard: Pasteboard) →  
Boolean
```

Returns true if the pasteboard contains a type that can be imported as tasks.

```
function pasteTasksFromPasteboard(pasteboard:  
Pasteboard) → Array of Task
```

Reads the most relevant pasteboard type and imports them as tasks. The tasks should then be moved to the desired destination.

Instance Properties

```
var app → Application read-only
```

Returns the shared Application.

```
var canRedo → Boolean read-only
```

Returns true if there are redoable actions.

```
var canUndo → Boolean read-only
```

Returns true if there are undoable actions.

var `console` → [Console](#) *read-only*

Returns the shared Console.

var `document` → [DatabaseDocument](#) or null *read-only*

var `flattenedFolders` → [FolderArray](#) *read-only*

Returns a flat array of all folders in the database, sorted by their order in the database.

var `flattenedProjects` → [ProjectArray](#) *read-only*

Returns a flat array of all projects in the database, sorted by their order in the database.

var `flattenedSections` → [SectionArray](#) *read-only*

Returns a flat array of all folders and project in the database, sorted by their order in the database.

var `flattenedTags` → [TagArray](#) *read-only*

Returns a flat array of all tags in the database, sorted by their order in the database.

var `flattenedTasks` → [TaskArray](#) *read-only*

Returns a flat array of all tasks in the database, including inbox items, root tasks for projects, task groups and individual tasks. Tasks are sorted by their order in the database, with the inbox preceeding the library.

var `folders` → [FolderArray](#) *read-only*

Returns the top-level folders in the database.

var `inbox` → [Inbox](#) *read-only*

Returns a copy of the Tasks currently in the inbox.

var `library` → [Library](#) *read-only*

Returns the top-level folders and projects in the database.

var `projects` → [ProjectArray](#) *read-only*

Returns the top-level folders in the database.

var `settings` → [Settings](#) *read-only*

var `tags` → [Tags](#) *read-only*

Returns the top-level tags in the database.

DatabaseObject

Instance Properties

var `id` → [ObjectIdentifier](#) *read-only*

Returns the identifier for this object.

DatedObject : DatabaseObject

Instance Properties

var `added` → Date or null

Returns the date the object was first saved. If the object is newly inserted, this will be null. For newly inserted objects, the added property may be set (but once an object is saved for the first time, the property is read-only).

var `modified` → Date or null

Returns the date the object was most recently modified. If the object is newly inserted, this will be null. For newly inserted objects, the modified property may be set (but once an object is saved for the first time, the property is read-only).

ActiveObject : DatedObject

Instance Properties

var `active` → Boolean

If true, then this object is considered active, otherwise the object is considered dropped.

var `effectiveActive` → Boolean *read-only*

Returns true if this object and all its containers are active.

Folder : [ActiveObject](#)

Class Functions

function [byIdentifier](#)(identifier: String) → [Folder](#) or null

Returns the Folder with the specified identifier, or null if no such folder exists.

Constructors

new [Folder](#)(name: String, position: [Folder](#) or [Folder.ChildInsertionLocation](#) or null) → [Folder](#)

Instance Functions

function [folderNamed](#)(name: String) → [Folder](#) or null

Returns the first child Folder with the given name that is contained directly in this folder, or null.

function [projectNamed](#)(name: String) → [Project](#) or null

Returns the first child Project of this folder with the given name, or null.

function [sectionNamed](#)(name: String) → [Project](#) or [Folder](#) or null

Returns the first child Folder or Project in this folder with the given name, or null.

```
function childNamed(name: String) → Project or Folder or null
```

An alias for sectionNamed.

```
function apply(function: Function) → ApplyResult or null
```

Calls the given function for this Folder and recursively into any child folders and projects. The tasks within any projects are not descended into.

Instance Properties

```
var after → Folder.ChildInsertionLocation read-only
```

Returns a location referring to the position just after this folder within its containing Folder or Database.

```
var before → Folder.ChildInsertionLocation read-only
```

Returns a location referring to the position just before this folder within its containing Folder or Database.

```
var beginning → Folder.ChildInsertionLocation read-only
```

Returns a location referring to the the beginning of the contained projects and folders in this folder.

```
var children → SectionArray read-only
```

An alias for sections.

var `ending` → [Folder.ChildInsertionLocation](#) ***read-only***

Returns a location referring to the the ending of the contained projects and folders in this folder.

var `flattenedChildren` → [SectionArray](#) ***read-only***

An alias for flattenedSections.

var `flattenedFolders` → [FolderArray](#) ***read-only***

Returns a flat array of all folders in this folder, sorted by their order in the database.

var `flattenedProjects` → [ProjectArray](#) ***read-only***

Returns a flat array of all projects in this folder, sorted by their order in the database.

var `flattenedSections` → [SectionArray](#) ***read-only***

Returns a flat array of all folders and project in this folder, sorted by their order in the database.

var `folders` → [FolderArray](#) ***read-only***

Returns the folders contained directly as children of this folder.

var `name` → String

The name of the folder.

var `parent` → [Folder](#) or null *read-only*

The parent Folder which contains this folder.

var `projects` → [ProjectArray](#) *read-only*

Returns the projects contained directly as children of this folder.

var `sections` → [SectionArray](#) *read-only*

Returns a sorted list of the folders and projects contained directly within this folder.

var `status` → [Folder.Status](#)

The folder's status.

Tag : [ActiveObject](#)

Class Functions

function `byIdentifier`(identifier: String) → [Tag](#) or null

Returns the Tag with the specified identifier, or null if no such tag exists.

Class Properties

var `forecastTag` → [Tag](#) or null *read-only*

The Forecast Tag, if it is set.

Constructors

`new Tag(name: String, position: Tag or Tag.ChildInsertionLocation or null) → Tag`

Instance Functions

`function tagNamed(name: String) → Tag or null`

Returns the first child Tag with the given name that is contained directly in this tag, or null.

`function childNamed(name: String) → Tag or null`

An alias for `tagNamed`.

`function apply(function: Function) → ApplyResult or null`

Calls the given function for this Tag and recursively into any child tags.

Instance Properties

`var after → Tag.ChildInsertionLocation read-only`

Returns a location referring to the position just after this tag.

`var allowsNextAction → Boolean`

If set and the tag is active, tasks with this tag applied cannot be the next task of a project.

var `availableTasks` → [TaskArray](#) *read-only*

Returns a sorted list of the tasks associated with this tag that are currently available. Recent changes may not be reflected until a `cleanup` is performed on the database.

var `before` → [Tag.ChildInsertionLocation](#) *read-only*

Returns a location referring to the position just before this tag.

var `beginning` → [Tag.ChildInsertionLocation](#) *read-only*

Returns a location referring to the beginning of the contained tags in this tag.

var `children` → [TagArray](#) *read-only*

An alias for tags.

var `ending` → [Tag.ChildInsertionLocation](#) *read-only*

Returns a location referring to the ending of the contained tags in this tag.

var `flattenedChildren` → [TagArray](#) *read-only*

An alias for flattenedTags.

var `flattenedTags` → [TagArray](#) *read-only*

Returns a flat array of all tags contained within this tag. Tags are sorted by their order in the database.

var **name** → String

var **parent** → [Tag](#) or null **read-only**

The parent Tag which contains this tag.

var **projects** → [ProjectArray](#) **read-only**

A convenience property that returns only Projects for the root tasks associated with this Tag.

var **remainingTasks** → [TaskArray](#) **read-only**

Returns a sorted list of the tasks associated with this tag that remaining to be completed. Recent changes may not be reflected until a `cleanup` is performed on the database.

var **status** → [Tag.Status](#)

The current status of the tag as a whole, which is derived from `allowsNextAction` and `active`.

var **tags** → [TagArray](#) **read-only**

Returns a sorted list of the tags contained directly within this tag, sorted by their library order.

var tasks → [TaskArray](#) *read-only*

Returns a sorted list of the tasks associated with this tag.

Task : [ActiveObject](#)

Class Functions

function [byParsingTransportText](#)(text: String, singleTask: Boolean or null) → Array of [Task](#)

Returns an array of tasks by parsing the transport text formatted input. Optionally, only the first task can be requested (but will still be returned in an array).

function [byIdentifier](#)(identifier: String) → [Task](#) or null

Returns the Task with the specified identifier, or null if no such task exists.

Constructors

new [Task](#)(name: String, position: [Project](#), [Task](#), or [Task.ChildInsertionLocation](#) or null) → [Task](#)

Returns a new Task at the given location. If a project or task is given as a location, the new task is placed at the end of the children of that parent. If no location is specified, then the task is created at the end of the inbox.

Instance Functions

function `taskNamed(name: String) → Task or null`

Returns the first child Task with the given name that is contained directly in this task, or null.

function `childNamed(name: String) → Task or null`

An alias for `taskNamed`.

function `appendStringToNote(stringToAppend: String)`

Appends `stringToAppend` to the end of the Task's note.

function `addLinkedFileURL(url: URL)`

Links a file URL to this task. In order to be considered a file URL, `url` must have the file scheme. That is, `url` must be of the form `file://path-to-file`. The file at `url` will not be added to database, rather a bookmark leading to it will be added. In order to add files to a task, use the `addAttachment` function. Linking files is especially useful for large files, as including large files in the database can degrade app performance.

function `removeLinkedFileWithURL(url: URL)`

Removes the first link to a file with the given `url`. This removes the bookmark that leads to the file at `url`. If the file itself is present in the database, use the `removeAttachmentAtIndex` function instead.

function `addAttachment(attachment: FileWrapper)`

Adds attachment as an attachment to the task. If the attachment is large, consider using the `addLinkedFileURL` function instead. Including large attachments in the database can degrade app performance.

`function removeAttachmentAtIndex(index: Number)`

Removes the attachment at `index` from this task's `attachments` array.

`function addTag(tag: Tag)`

Adds a `Tag` to this task, appending it to the end of the list of associated tags. If the tag is already present, no change is made. The Database function `moveTags` can be used to control the ordering of tags within the task.

`function addTags(tags: Array of Tag)`

Adds multiple `Tags` to this task, appending them to the end of the list of associated tags. For any tags already associated with the `Task`, no change is made. The Database function `moveTags` can be used to control the ordering of tags within the task.

`function removeTag(tag: Tag)`

Removes a `Tag` from this task. If the tag is not associated with this task, no change is made.

`function removeTags(tags: Array of Tag)`

Removes multiple `Tags` from this task. If a tag is not associated with this task, no change is made.

`function clearTags()`

Removes multiple Tags from this task. If a tag is not associated with this task, no change is made.

`function markComplete(date: Date or null) → Task`

If the task is not completed, marks it as complete with the given completion date (or the current date if no date is specified). For repeating tasks, this makes a clone of the task and marks that clone as completed. In either case, the task that has been marked completed is returned.

`function markIncomplete()`

If the task is completed, marks it as incomplete.

`function drop(allOccurrences: Boolean)`

Drops this Task. If true is passed in for allOccurrences then this task will not repeat, even if it has a repetitionRule set on it. If false is passed in for allOccurrences, this task will repeat as normal.

`function apply(function: Function) → ApplyResult or null`

Calls the given function for this Task and recursively into any child task.

`function addNotification(info: Number or Date) → Task.Notification`

Add a notification from the specification in info. Supplying a Date creates an absolute notification that will fire at that date. Supplying a Double will create a due-

relative notification. Specifying a due relative notification when this task's effectiveDueDate is not set will result in an error.

```
function removeNotification\(notification:  
Task.Notification\)
```

Remove an active notification for this task. Supplying a notification that is not in this task's notifications array, or a notification that has task to something other than this task results in an error.

Instance Properties

```
var after → Task.ChildInsertionLocation read-only
```

The location after this task within its parent task's children. If this task has no parent task, then this is the position adjacent to it in its container.

```
var assignedContainer → Project, Task, or Inbox or null
```

For tasks in the inbox, the tentatively assigned project or parent task, which will be applied on cleanup.

```
var attachments → Array of FileWrapper
```

An array of FileWrapper objects representing the attachments associated with the task.

```
var before → Task.ChildInsertionLocation read-only
```

The location before this task within its parent task's children. If this task has no parent task, then this is the position adjacent to it in its container.

var **beginning** → [Task.ChildInsertionLocation](#) *read-only*

The location at the beginning of this task's children.

var **children** → [TaskArray](#) *read-only*

An alias for tasks.

var **completed** → Boolean *read-only*

True if the task has been marked completed. Note that a task may be effectively considered completed if a containing task is marked completed.

var **completedByChildren** → Boolean

If set, the Task will be automatically marked completed when its last child Task is marked completed.

var **completionDate** → Date **or null** *read-only*

If set, the Task is completed.

var **containingProject** → [Project](#) **or null** *read-only*

The Project that this Task is contained in, either as the root of the project or indirectly from a parent task. If this task is in the inbox, then this will be null.

var **deferDate** → Date **or null**

If set, the Task is not actionable until this date.

var **dropDate** → Date **or null** ***read-only***

If set, the Task is dropped.

var **dueDate** → Date **or null**

If set, the Task should be completed by this date.

var **effectiveCompletedDate** → Date **or null** ***read-only***

Returns the computed effective completion date for the Task, based on its local `completionDate` and those of its containers.

var **effectiveDeferDate** → Date **or null** ***read-only***

Returns the computed effective defer date for the Task, based on its local `deferDate` and those of its containers.

var **effectiveDropDate** → Date **or null** ***read-only***

Returns the computed effective drop date for the Task, based on its local `dropDate` and those of its containers.

var **effectiveDueDate** → Date **or null** ***read-only***

Returns the computed effective due date for the Task, based on its local `dateDue` and those of its containers.

var `effectiveFlagged` → Boolean *read-only*

Returns the computed effective flagged status for the Task, based on its local flagged and those of its containers.

var `ending` → Task.ChildInsertionLocation *read-only*

The location at the end of this task's children.

var `estimatedMinutes` → Number or null

The estimated number of minutes this task will take to finish, or null if no estimate has been made.

var `flagged` → Boolean

The flagged status of the task.

var `flattenedChildren` → TaskArray *read-only*

An alias for flattenedTasks.

var `flattenedTasks` → TaskArray *read-only*

Returns a flat array of all tasks contained within this task. Tasks are sorted by their order in the database.

var `hasChildren` → Boolean *read-only*

Returns true if this task has children, more efficiently than checking if children is empty.

var `inInbox` → Boolean ***read-only***

True if the task is a direct child of the inbox, but not if the task is contained by another task that is in the inbox.

var `linkedFileURLs` → Array of URL ***read-only***

The list of file URLs linked to this task. The files at these URLs are not present in the database, rather the database holds bookmarks leading to these files. These links can be read on iOS, but not written to.

var `name` → String

The title of the task.

var `note` → String

The tasks's note.

var `notifications` → Array of Task.Notification ***read-only***

An array of the notifications that are active for this task.

var `parent` → Task or null ***read-only***

The parent Task which contains this task.

var `project` → Project or null ***read-only***

The Project that this Task is the root task of, or `null` if this task is in the inbox or contained by another task.

`var repetitionRule → Task.RepetitionRule or null`

The object holding the repetition properties for this task, or null if it is not repeating.

`var sequential → Boolean`

If `true`, then children of this task form a dependency chain. For example, the first task blocks the second one until the first is completed.

`var shouldUseFloatingTimeZone → Boolean`

When set, the `dueDate` and `deferDate` properties will use floating time zones.

(Note: if a Task has no due or defer dates assigned, this property will revert to the database's default setting.)

`var tags → TagArray read-only`

Returns the Tags associated with this Task.

`var taskStatus → Task.Status read-only`

Returns the current status of the task.

`var tasks → TaskArray read-only`

Returns all the tasks contained directly in this task, sorted by their library order.

Perspective.Custom : DatedObject

Class Functions

function `byName(name: String)` → Perspective.Custom or `null`

A custom perspective with the given name, if one exists. If there are multiple perspectives with the same name, it is not defined which will be returned.

function `byIdentifier(identifier: String)` → Perspective.Custom or `null`

Returns the custom perspective with the specified identifier, or `null` if no such perspective exists.

Class Properties

var `all` → Array of Perspective.Custom *read-only*

Returns all the custom perspectives.

Instance Functions

function `fileWrapper()` → FileWrapper

Returns an archived file wrapper for the custom perspective. The file wrapper's preferred filename will be the name of the perspective with an appropriate file extension applied. Its contents will include a plist representing the perspective's

settings, along with any image attachments needed to display its icon.

function

`writeFileRepresentationIntoDirectory(parentURL: URL) → URL`

Writes the perspective's `fileWrapper()` within a given parent directory URL, returning the URL of the saved FileWrapper. This function requires sandboxed access to the parent folder; it may be easier to work with the perspective's `fileWrapper()`, which can be accessed directly or saved to disk using `FileSaver`.

Instance Properties

var `identifier` → String *read-only*

The unique identifier of the custom perspective.

var `name` → String *read-only*

The name of the custom perspective.

Task.Notification : [DatedObject](#)

Instance Properties

var `absoluteFireDate` → Date

The absolute date at which this notification will fire, if its `kind` is `absolute`. Getting or setting this property throws an error if this notification's `kind` is not `absolute`.

var `initialFireDate` → Date ***read-only***

The time at which this notification will fire. For due or defer-relative notifications, this date will change with its task object's due and defer dates.

var `isSnoozed` → Boolean ***read-only***

Whether or not this notification has been snoozed.

var `kind` → Task.Notification.Kind ***read-only***

This notification's kind. A kind of unknown indicates that the notification is in an invalid state.

var `nextFireDate` → Date **or null** ***read-only***

The next time at which this notification will fire. This will only have a value if the `initialFireDate` is not yet reached, or this notification's `repeatInterval` is greater than 0.

var `relativeFireOffset` → Number

The relative offset in minutes at which this notification will fire from the specified date on its task. Getting or setting this property throws an error if this notification's `kind` is not either `dueRelative` or `deferRelative`.

var `repeatInterval` → Number

How often in seconds this notification will fire once its `initialFireDate` is reached. Setting this to 0 or any negative number will cease repetition of this notification.

`var task → Task or null read-only`

The Task object this notification will fire for.

`var usesFloatingTimeZone → Boolean read-only`

Whether or not the notification's fire date uses floating time zones. This can only return true if the notification's kind is absolute. This can be changed by `settings.shouldUseFloatingTimeZone` on this notification's task

Project : [DatabaseObject](#)

Class Functions

`function byIdentifier(identifier: String) → Project or null`

Returns the Project with the specified identifier, or null if no such project exists.

Constructors

`new Project(name: String, position: Folder or Folder.ChildInsertionLocation or null) → Project`

Instance Functions

function `taskNamed`(name: String) → [Task](#) or null

Returns the first top-level Task in this project the given name, or null.

function `appendStringToNote`(stringToAppend: String)

Appends stringToAppend to the end of the Project's root Task's note.

function `addAttachment`(attachment: [FileWrapper](#))

Adds attachment as an attachment to the Project's root Task. If the attachment is large, consider using the `addLinkedFileURL` function instead. Including large attachments in the database can degrade app performance.

function `removeAttachmentAtIndex`(index: Number)

Removes the attachment at index from this Project's root Task's attachments array.

function `markComplete`(date: Date or null) → [Task](#)

If the project is not completed, marks it as complete with the given completion date (or the current date if no date is specified). For repeating project, this makes a clone of the project and marks that clone as completed. In either case, the project that has been marked completed is returned.

function `markIncomplete`()

If the project is completed, marks it as incomplete.

`function addNotification(info: Number or Date) → Task.Notification`

Add a notification to the project from the specification in `info`. Supplying a `Date` creates an absolute notification that will fire at that date. Supplying a `Double` will create a due-relative notification. Specifying a due-relative notification when this project's task's `effectiveDueDate` is not set will result in an error.

`function removeNotification(notification: Task.Notification)`

Remove an active notification for this project. Supplying a notification that is not in this task's `notifications` array, or a notification that has task to something other than this project's task results in an error.

`function addTag(tag: Tag)`

Adds a `Tag` to this project, appending it to the end of the list of associated tags. If the tag is already present, no change is made. The Database function `moveTags` can be used to control the ordering of tags within the task.

`function addTags(tags: Array of Tag)`

Adds multiple `Tags` to this project, appending them to the end of the list of associated tags. For any tags already associated with the `Task`, no change is made. The Database function `moveTags` can be used to control the ordering of tags within the task.

`function removeTag(tag: Tag)`

Removes a `Tag` from this project. If the tag is not associated with this project, no change is made.

`function removeTags(tags: Array of Tag)`

Removes multiple Tags from this project. If a tag is not associated with this project, no change is made.

`function clearTags()`

Removes multiple Tags from this project. If a tag is not associated with this project, no change is made.

`function addLinkedFileURL(url: URL)`

Links a file URL to this task. In order to be considered a file URL, url must have the file scheme. That is, url must be of the form file://path-to-file. The file at url will not be added to database, rather a bookmark leading to it will be added. In order to add files to a task, use the addAttachment function. Linking files is especially useful for large files, as including large files in the database can degrade app performance. This function throws an error if invoked on iOS.

`function removeLinkedFileWithURL(url: URL)`

Removes the first link to a file with the given url. This removes the bookmark that leads to the file at url. If the file itself is present in the database, use the removeAttachmentAtIndex function instead.

Instance Properties

`var after → Folder.ChildInsertionLocation read-only`

Returns a location referring to the position just after this project within its containing Folder or Database.

var **attachments** → Array of [FileWrapper](#)

An array of FileWrapper objects representing the attachments associated with the Project's root Task.

var **before** → [Folder.ChildInsertionLocation](#) **read-only**

Returns a location referring to the position just before this project within its containing Folder or Database.

var **beginning** → [Task.ChildInsertionLocation](#) **read-only**

Returns a location referring to the position after the last Task directly contained in the root task of this project.

var **children** → [TaskArray](#) **read-only**

An alias for tasks.

var **completed** → Boolean **read-only**

True if the project has been marked completed. Note that a project may be effectively considered completed if a containing project is marked completed.

var **completedByChildren** → Boolean

If set, the project will be automatically marked completed when its last child Task is marked completed.

var **completionDate** → Date **or null**

If set, the project is completed.

var **containsSingletonActions** → Boolean

If set to `true`, the project contains single tasks, and has no next task.

var **defaultSingletonActionHolder** → Boolean

If set to `true`, this is the Project that inbox tasks that have enough information specified (as selected by the user's preferences) will be filed into upon a clean-up.

var **deferDate** → Date **or null**

If set, the project is not actionable until this date.

var **dropDate** → Date **or null**

If set, the project is dropped.

var **dueDate** → Date **or null**

If set, the project should be completed by this date.

var **effectiveCompletedDate** → Date **or null** *read-only*

Returns the computed effective completion date for the Project, based on its local completionDate and those of its containers.

var effectiveDeferDate → Date or null **read-only**

Returns the computed effective defer date for the Project, based on its local deferDate and those of its containers.

var effectiveDropDate → Date or null **read-only**

Returns the computed effective drop date for the Project, based on its local dropDate and those of its containers.

var effectiveDueDate → Date or null **read-only**

Returns the computed effective due date for the Project, based on its local dateDue and those of its containers.

var effectiveFlagged → Boolean **read-only**

Returns the computed effective flagged status for the Project, based on its local flagged and those of its containers.

var ending → [Task.ChildInsertionLocation](#) **read-only**

Returns a location referring to the position before the first Task directly contained in the root task of this project.

var estimatedMinutes → Number or null

The estimated number of minutes this Project will take to finish, or null if no estimate has been made.

var **flagged** → Boolean

The flagged status of the project.

var **flattenedChildren** → TaskArray **read-only**

An alias for flattenedTasks.

var **flattenedTasks** → TaskArray **read-only**

Returns a flat array of all tasks contained within this Project's root Task. Tasks are sorted by their order in the database.

var **hasChildren** → Boolean **read-only**

Returns true if this Project's root Task has children, more efficiently than checking if children is empty.

var **lastReviewDate** → Date or null

The date on which the last review was performed. See also nextReviewDate.

var **linkedFileURLs** → Array of URL **read-only**

The list of file URLs linked to this project's root task. The files at these URLs are not present in the database, rather the database holds bookmarks leading to these

files. These links can be read on iOS, but not written to.

var **name** → String

The name of the Projects root task.

var **nextReviewDate** → Date **or null**

The scheduled date for the next review. See also nextReviewDate.

var **nextTask** → [Task](#) **or null** *read-only*

Returns the very next task that can be completed in the project, or `null` if there is none or the project contains singleton actions.

var **note** → String

The Project's root Task's note.

var **notifications** → Array of [Task.Notification](#) *read-only*

An array of the notifications that are active for this project.

var **parentFolder** → [Folder](#) **or null** *read-only*

The Folder which contains this project.

var **repetitionRule** → [Task.RepetitionRule](#) **or null**

The object holding the repetition properties for this project, or null if it is not repeating.

var `reviewInterval` → [`Project.ReviewInterval`](#)

The object holding the review repetition properties for this project. See also `lastReviewDate` and `nextReviewDate`.

var `sequential` → Boolean

If `true`, then children of this project form a dependency chain. For example, the first task blocks the second one until the first is completed.

var `shouldUseFloatingTimeZone` → Boolean

When set, the `dueDate` and `deferDate` properties will use floating time zones. (Note: if a `Project` has no due or defer dates assigned, this property will revert to the database's default setting.)

var `status` → [`Project.Status`](#)

The current status of the project as a whole. This does not reflect the status of individual tasks within the project root task – a project may be marked with the `Done` status and its individual tasks will be left with the completion state they had, in case the status is changed again to `Active`.

var `tags` → [`TagArray`](#) *read-only*

Returns the Tags associated with this `Project`.

var `task` → [Task](#) *read-only*

Returns the root task of the project, which holds the bulk of the project information, as well as being the container for tasks within the project. If you wish to copy the project or move it to a location that requires tasks, you would use this task as the object to be copied or moved.

var `taskStatus` → [Task.Status](#) *read-only*

Returns the current status of the project.

var `tasks` → [TaskArray](#) *read-only*

Returns all the tasks contained directly in this Project's root Task, sorted by their library order.

DateComponents

Constructors

new `DateComponents()` → [DateComponents](#)

Instance Properties

var `date` → Date **or null** *read-only*

var `day` → Number **or null**

var `era` → Number **or null**

var `hour` → Number **or null**

var `minute` → Number **or** null

var `month` → Number **or** null

var `nanosecond` → Number **or** null

var `second` → Number **or** null

var `timeZone` → [TimeZone](#) **or** null

var `year` → Number **or** null

DateRange

Instance Properties

var `end` → Date *read-only*

var `name` → String *read-only*

var `start` → Date *read-only*

Decimal

The `Decimal` class provides support for operating on base-10 numbers, which may not be exactly representable by types like the built-in JavaScript `Number` class. Note that `Decimal` does not use the built-in arithmetic operations; for example, to add two `Decimal` instances, you must use the `add()` function.

Class Functions

function `fromString`(string: String) → [Decimal](#)

Parses the given string into a `Decimal`. If the string cannot be parsed, `notANumber` is returned.

Class Properties

var `maximum` → `Decimal` *read-only*

Returns the maximum representable `Decimal` value.

var `minimum` → `Decimal` *read-only*

Returns the minimum representable `Decimal` value.

var `notANumber` → `Decimal` *read-only*

Returns a `Decimal` that represents a non-number value. Any arithmetic operations involving non-number values will return `notANumber`.

var `one` → `Decimal` *read-only*

Returns a `Decimal` representing one.

var `zero` → `Decimal` *read-only*

Returns a `Decimal` representing zero.

Instance Functions

function `toString()` → `String`

Converts the `Decimal` to a `String` representation.

function `add`(number: `Decimal`) → `Decimal`

Generates a new `Decimal` by adding the argument and the receiver.

function `subtract`(number: `Decimal`) → `Decimal`

Generates a new `Decimal` by subtracting the argument from the receiver.

function `multiply`(number: `Decimal`) → `Decimal`

Generates a new `Decimal` by multiplying the argument and the receiver.

function `divide`(number: `Decimal`) → `Decimal`

Generates a new `Decimal` by dividing the receiver by the argument.

function `compare`(number: `Decimal`) → `Number`

Compares the receiver and argument. If the receiver is less than the argument, `-1` is returned. If the receiver is greater than the argument, `1` is returned. Otherwise, `0` is returned. `NaN` is considered less than any valid number, and equal to itself.

function `equals`(number: `Decimal`) → `Boolean`

Returns `true` if the receiver and argument represent the same number (or both are `NaN`), and `false` otherwise.

Device

Class Properties

var `current` → [Device](#) *read-only*

The device the current application is running on.

Instance Properties

var `iOS` → Boolean *read-only*

A convenience that returns `true` on iPhone and iPad devices

var `iPad` → Boolean *read-only*

A convenience that returns `true` only on iPad devices, but not on iPhone devices.

var `mac` → Boolean *read-only*

A convenience that returns `true` only on Mac devices.

var `operatingSystemVersion` → [Version](#) *read-only*

The current operation system version running on the device

var `type` → [DeviceType](#) or null *read-only*

The general type of the current device

DeviceType

Class Properties

var `all` → Array of [DeviceType](#) *read-only*

var `iPad` → [DeviceType](#) *read-only*

An iPad

var `iPhone` → [DeviceType](#) *read-only*

An iPhone

var `mac` → [DeviceType](#) *read-only*

A Mac device

Document

Class Functions

function `makeNew`(resultFunction: [Function](#)(document: [Document](#) or [Error](#)) or null) → [Promise](#) of [Document](#)

Create a new document, which can be populated with data and then presented. On iOS, if the document is not presented by the time the `resultFunction` returns, it will be closed. On macOS, the document will be left around and accessible to the running script. `resultFunction` is executed before any functions tethered to the result Promise are executed. Returns a Promise that will yield the new document

or an error.

```
function makeNewAndShow(resultFunction:  
Function(document: Document or Error) or null) →  
Promise of Document
```

Create a new document and presents it. Returns a Promise that will yield the new document or an error.

Instance Functions

```
function close(didCancel: Function(document: Document)  
or null)
```

Close this document. If for some reason the document cannot be closed, the `didCancel` function may be called at some point in the future, with the original document as the single argument. For example, on the Mac the user may review unsaved changes and may cancel the close operation. If the document is closed, the `didCancel` function will not be called at all.

```
function save()
```

Save this document.

```
function fileWrapper(type: String or null) →  
FileWrapper
```

Deprecated: Please use `makeFileWrapper()` instead. Returns a new `FileWrapper` representing the contents of the document formatted as the specified type, or its current `fileType` if a `null` is passed for the type.

function `makeFileWrapper`(baseName: String, type: String or null) → [Promise](#) of [FileWrapper](#)

Generates a `FileWrapper` representing the contents of the document formatted as the specified type, or its current `fileType` if a `null` is passed for the type. Returns a `Promise` that will yield the file wrapper or an error. The returned file wrapper will have a name based off the given `baseName` and the default path extension for the requested file type.

function `undo`()

Undo the last done action.

function `redo`()

Redo the last undone action.

function `show`(completed: [Function](#)() or null)

Presents the document, ordering the window forward on macOS, and possibly closing the existing document and opening the new one on iOS. Calls the completion function once the document is shown.

Instance Properties

var `canRedo` → Boolean *read-only*

Whether there are currently any actions that can be redone.

var `canUndo` → Boolean *read-only*

Whether there are currently any actions that can be undone.

var `fileType` → String **or** null *read-only*

The file type identifier the document uses when saving, if set.

var `name` → String **or** null *read-only*

Document name.

var `writableTypes` → Array **of** String *read-only*

A list of all of the file types that this document can be written as.

DatabaseDocument : Document

Instance Functions

function `newWindow()` → Promise **of** DocumentWindow

Returns a Promise that will yield either a newly created and displayed Window or an error. On macOS, this method respects the System Preference governing new window behavior (tab vs. window). That preference is accessible at System Preferences > Dock > Prefer tabs when opening documents.

function `newTabOnWindow(window: DocumentWindow)` → Promise **of** DocumentWindow

Returns a Promise that will yield either a new tab adjacent to window or an error. This is not available on iOS.

Instance Properties

var `windows` → Array of [DocumentWindow](#) *read-only*

Email

A set of parameters for generating an email.

Constructors

new `Email()` → [Email](#)

Instance Functions

function `generate()`

Presents the generated email to the user for them to send (or discard). On iOS, any included attachment FileWrappers that are directories will be converted to Zip files.

Instance Properties

var `blindCarbonCopy` → String or Array of String or null

var `body` → String or null

var `carbonCopy` → String or Array of String or null

var `fileWrappers` → Array of [FileWrapper](#)

var `receiver` → String **or** Array **of** String **or** null

var `subject` → String **or** null

Error

The built-in JavaScript [Error](#) constructor.

Instance Properties

var `causedByUserCancelling` → Boolean *read-only*

Returns true for errors that are caused by the user cancelling an operation. For example, if the user selects the Cancel button in a `FilePicker`, the Promise will signal an error that reflects this.

FilePicker

A `FilePicker` allows the user to select URLs for files via the system-supplied file picking interface.

Constructors

new `FilePicker()` → [FilePicker](#)

Returns a new `FilePicker` with default settings.

Instance Functions

function `show()` → [Promise](#) of Array of [URL](#)

Presents the system file selection interface to the user, allowing them to choose one or more files of the given types. The returned Promise will yield the chosen URLs on success. If the user cancels choosing, the Promise will be rejected. Note that even when picking a single file or folder, the result will be an array of URLs.

Instance Properties

var `folders` → Boolean

If `true`, then folders may be selected, but not files. In this case, `types` is ignored. Defaults to `false`.

var `message` → String

A message to display describing what files are being picked. This is currently only supported on macOS.

var `multiple` → Boolean

If `true`, then multiple files may be selected. Defaults to `false`.

var `types` → Array of [TypeIdentifier](#) or `null`

The file types that will be allowed. If `null`, all file types will be allowed. Defaults to `null`.

FileSaver

A `FileSaver` allows the user to save a `FileWrapper` to a URLs via the system-supplied file picking interface.

Constructors

`new FileSaver()` → [`FileSaver`](#)

Returns a new `FileSaver` with default settings.

Instance Functions

`function show(fileWrapper: FileWrapper)` → [`Promise of URL`](#)

Presents the system file saving interface to the user, allowing them to choose a location and file name to save the file wrapper. The returned `Promise` will yield the chosen URL on success. If the user cancels choosing, the `Promise` will be rejected.

Instance Properties

`var message` → `String`

A message to display describing what file is being saved. This is currently only supported on macOS.

`var nameLabel` → `String`

The label shown next to the user-editable file name field. This is currently only supported on macOS.

`var prompt` → `String`

The prompt shown on the the save button. This is currently only supported on macOS.

var `types` → Array of [TypeIdentifier](#) or null

The file types that will be allowed. If null, all file types will be allowed. Defaults to null.

FileWrapper

Class Functions

function `withContents`(name: String or null, contents: [Data](#)) → [FileWrapper](#)

Returns a new `FileWrapper` that represents a flat file containing the given data.

function `withChildren`(name: String or null, children: Array of [FileWrapper](#)) → [FileWrapper](#)

Returns a new `FileWrapper` that represents a directory with the given child file wrappers. Each child file wrapper must have a unique name specified.

function `fromURL`(url: [URL](#), options: Array of [FileWrapper.ReadingOptions](#) or null) → [FileWrapper](#)

Reads a `FileWrapper` from an existing URL.

Instance Functions

function `childNamed`(name: String) → [FileWrapper](#) or null

Returns the child file wrapper with the specified name, or null if the receiver is not a directory or doesn't have a child with that name.

function `filenameForChild`(child: [FileWrapper](#)) → String or null

Returns the unique file name that will be used for the given child FileWrapper, or null if this file wrapper is not a child of the receiver.

function `write`(url: [URL](#), options: Array of [FileWrapper.WritingOptions](#) or null, originalContentsURL: [URL](#) or null)

Writes the FileWrapper to the given URL. **NOTE: Any existing file or folder at the desination URL will be replaced. Care must be taken when developing scripts to avoid unintended data loss.**

Instance Properties

var `children` → Array of [FileWrapper](#) *read-only*

Returns an Array of child FileWrappers, if this represents a directory. Otherwise, an empty array is returned.

var `contents` → [Data](#) or null *read-only*

Returns the regular file contents of the wrapper, if this represents a regular file. Otherwise, null is returned.

var `destination` → [URL](#) or null *read-only*

Returns the destination if this represents a symbolic link. Otherwise, null is returned.

var `filename` → String or null

Returns the actual file name that was last read for this file wrapper. Depending on the names of other sibling wrappers, this may not be what file name will be written.

var `preferredFilename` → String or null

Returns the preferred file name that should be used when writing the file wrapper if no other file in the same parent directory wrapper is in use.

var `type` → [FileWrapper.Type](#) *read-only*

Returns the type of this FileWrapper.

FileWrapper.ReadingOptions

Class Properties

var `Immediate` → [FileWrapper.ReadingOptions](#) *read-only*

Whether the contents are read immediately, or (by default) as the file wrappers are accessed.

var `WithoutMapping` → [FileWrapper.ReadingOptions](#) *read-only*

Allow disabling file mapping.

var `all` → Array of [FileWrapper.ReadingOptions](#) *read-only*

FileWrapper.Type

Class Properties

var `Directory` → [FileWrapper.Type](#) *read-only*

A FileWrapper that represents a directory with zero or more child wrappers.

var `File` → [FileWrapper.Type](#) *read-only*

A FileWrapper that represents a regular file with data contents.

var `Link` → [FileWrapper.Type](#) *read-only*

A FileWrapper that represents a symbolic link to another location.

var `all` → Array of [FileWrapper.Type](#) *read-only*

FileWrapper.WritingOptions

Class Properties

var `Atomic` → [FileWrapper.WritingOptions](#) *read-only*

Write the entire `FileWrapper` atomically, so that either the entire file package is replaced or none of it is.

var `UpdateNames` → [`FileWrapper.WritingOptions`](#) *read-only*

On successful writing, update the filename of each file wrapper recursively so that following writes can use performance optimizations using hard links.

var `all` → Array of [`FileWrapper.WritingOptions`](#) *read-only*

Folder.ChildInsertionLocation

A location specified relative to an existing `Folder`, `Project`, or `Database`. These cannot be instantiated directly, rather they are returned from properties like `Folder.beginning`, `Project.before`, or `Database.ending`.

Folder.Status

Class Properties

var `Active` → [`Folder.Status`](#) *read-only*

The folder is active.

var `Dropped` → [`Folder.Status`](#) *read-only*

The folder has been dropped.

var `all` → Array of [`Folder.Status`](#) *read-only*

ForecastDay

An object representing one of the selectable days in the forecast perspective.

Class Properties

var `badgeCountsIncludeDeferredItems` → Boolean

Determines whether or not badges on Forecast days include items that are not yet available.

Instance Functions

function `badgeKind()` → [ForecastDay.Status](#)

The status of the badge on this forecast day.

Instance Properties

var `badgeCount` → Number *read-only*

The number of available tasks on this forecast day.

var `date` → Date *read-only*

The date this forecast day represents. If this day's kind is Past or DistantFuture the date returned will be years from the current time.

var `deferredCount` → Number *read-only*

The number of remaining deferred tasks on this forecast day.

var `kind` → [ForecastDay.Kind](#) *read-only*

var `name` → String *read-only*

ForecastDay.Kind

Class Properties

var `Day` → [ForecastDay.Kind](#) *read-only*

The node represents a specific day in the Forecast week or month grid.

var `DistantFuture` → [ForecastDay.Kind](#) *read-only*

The node represents all days more than a year from now.

var `FutureMonth` → [ForecastDay.Kind](#) *read-only*

The node represents a month within the next year.

var `Past` → [ForecastDay.Kind](#) *read-only*

The node represents all days in the past.

var `Today` → [ForecastDay.Kind](#) *read-only*

The node represents today.

var `all` → Array of [ForecastDay.Kind](#) *read-only*

ForecastDay.Status

Class Properties

var `Available` → [ForecastDay.Status](#) *read-only*

There is at least one available task on the node's day, but no task is due soon or overdue. The node's badgeCount is the number of available tasks.

var `DueSoon` → [ForecastDay.Status](#) *read-only*

There is at least one available task on the node's day, and at least one task due that day is due soon, but no tasks due that day are overdue. The node's badgeCount is the number of available tasks.

var `NoneAvailable` → [ForecastDay.Status](#) *read-only*

There are no available tasks on the node's day. The node's badgeCount is guaranteed to be zero.

var `Overdue` → [ForecastDay.Status](#) *read-only*

There is at least one available task on the node's day, and at least one task due that day is overdue. The node's badgeCount is the number of available tasks.

var `all` → Array of [ForecastDay.Status](#) *read-only*

Form

Form provides a mechanism to collect input from the user. Each form contains one or more instances of subclasses of `Field`, which are given a key. As the form is filled out, `values` object is populated with the values from the user interface.

Constructors

`new Form()` → [`Form`](#)

Instance Functions

function `addField(field: Form.Field, index: Number or null)`

Adds the new `Field` to the `Form`, at the indicated position, or at the end if no position is specified. If the field has a default value, it will be added to the `values` result object immediately.

function `removeField(field: Form.Field)`

Removes the `Field` from the `Form`. Any entry in `thevalues`` for this field will be removed as well.

function `show(title: String, confirmTitle: String) → Promise of Form`

Present the `Form` to the user, and return a `Promise` to be fulfilled or rejected when the user commits or cancels the form.

Instance Properties

var `fields` → Array of `Form.Field` *read-only*

The current `Field` instances in the form, which will be visible to the user entering input.

var `validate` → `Function`(**Form:** `Form`) → Boolean **or** null

A function to check whether the entered values are acceptable. The form to validate is passed as the argument and the function is expected to return a boolean result or null to perform default validation. If an `Error` is thrown, its message will be displayed in the form as the reason for validation failure. Note that the validation function may add or remove fields and update entries in the `values` object (which will cause the interface to be updated). This is called any time the user edits values, or a field is added or removed. If no `validate` function is specified or it returns null, some per-field default validation will be performed (see `Form.Field.Option`). If the `validate` function returns a boolean result, no default validation will be performed.

var `values` → `Object` *read-only*

An object with the collected values for each field, stored under the key for that field.

Form.Field

A single entry for a user input value in a `Form`. Each field can only be added to a single `Form`. This class cannot be constructed directly.

Instance Properties

var `displayName` → String **or** null *read-only*

Human readable string used as the label for this field.

var **key** → String *read-only*

Key to use when storing the value for this field in the containing form's values object.

Form.Field.Checkbox : Form.Field

Constructors

new Form.Field.Checkbox(key: String, displayName: String or null, value: Boolean or null) → Form.Field.Checkbox

Returns a new Checkbox field, optionally with an initial value (which will be false if no value is specified).

Form.Field.Date : Form.Field

Constructors

new Form.Field.Date(key: String, displayName: String or null, value: Date or null, formatter: Formatter.Date or null) → Form.Field.Date

Returns a new Date field, optionally with an initial value, and optionally a date formatter. If no formatter is specified, a default one will be created that follows the user's date formatting preferences to display and determine component ordering

when parsing dates. Relative dates like “1d”, “tomorrow”, “now” can also be entered.

Form.Field.MultipleOptions : Form.Field

Constructors

`new Form.Field.MultipleOptions(key: String, displayName: String or null, options: Array of Object, names: Array of String or null, selected: Array of Object) → Form.Field.MultipleOptions`

Returns a new `MultipleOptions` field, allowing the user to pick multiple items from a list of option objects. A list of names may also be given, which must have the same length as the options array if so. If no names are given, the objects are converted to strings for display. An array of zero or more initially selected objects (which must be members of the options array) may also be given. An empty array is valid input for the initially selected items. Additionally, it is valid for `MultipleOptions` fields to have a value that is an empty array.

Form.Field.Option : Form.Field

Constructors

`new Form.Field.Option(key: String, displayName: String or null, options: Array of Object, names: Array of`

String or null, selected: [Object](#) or null,
nullOptionTitle: String or null) → [Form.Field.Option](#)

Returns a new `Option` field, allowing the user to pick from a list of option objects. A list of names may also be given, which must have the same length as the options array if so. If no names are given, the objects are converted to strings for display. An initially selected object (which must be a member of the options array) may also be given. If the field is not configured to allow a `null` value and no initially selected value is specified, the user must select a value before the field is considered valid under the default form validation.

Instance Properties

var `allowsNull` → Boolean

If set to `true`, an option will be added to allow selecting `null`.

var `nullOptionTitle` → String or null

If `null` is allowed, this will be used for the title of that option. Otherwise a default title will be used.

`Form.Field.Password` : [Form.Field](#)

A field for text-based input, optionally using a `Formatter` to convert the string value into a different type.

Constructors

```
new Form.Field.Password(key: String, displayName:
String or null, value: String or null) →
Form.Field.Password
```

Returns a new Password field, optionally with an initial value. The displayed text will be obscured.

Form.Field.String : [Form.Field](#)

A field for text-based input, optionally using a Formatter to convert the string value into a different type.

Constructors

```
new Form.Field.String(key: String, displayName: String
or null, value: Object or null, formatter: Formatter
or null) → Form.Field.String
```

Returns a new String field, optionally with an initial value and formatter. If a formatter is specified, the value should be of the output type from the formatter or null. If no formatter is specified, the value should be a string or null.

Formatter

Formatter.Date : [Formatter](#)

Class Functions

```
function withStyle(dateStyle: Formatter.Date.Style,  
timeStyle: Formatter.Date.Style or null) →  
Formatter.Date
```

A formatter that will display dates according to the specified date and time formats selected in system settings.

```
function withFormat(format: String) → Formatter.Date
```

Returns a formatter with a specific ICU date format and the user's current locale, calendar, and timeZone. See <http://userguide.icu-project.org/formatparse/datetime/> for details on date format strings.

Class Properties

```
var iso8601 → Formatter.Date read-only
```

Return a date formatter that produces ISO–8601 formatted dates, using the Gregorian calendar and the UTC time zone.

Instance Functions

```
function stringFromDate(date: Date) → String
```

```
function dateFromString(string: String) → Date or null
```

Instance Properties

```
var calendar → Calendar
```

```
var dateFormat → String read-only
```

var `locale` → [Locale](#)

var `timeZone` → [TimeZone](#)

Formatter.Decimal: [Formatter](#)

This formatter class formats and parses Decimal-valued strings (note, *not* Number values).

Class Functions

function `currency`(code: String or null) → [Formatter.Decimal](#)

Returns a new formatter that will display the value as a currency value. An ISO currency code may be specified to pick a specific currency, or null may be passed to use the default currency for the user's locale. If the argument is not a valid currency code, an error will be thrown.

Class Properties

var `currencyCodes` → Array of String ***read-only***

Deprecated: Please use the `currencyCode` property on `Locale` instead. Returns the list of known ISO currency codes

var `custom` → [Formatter.Decimal](#) ***read-only***

Returns a new formatter that can be configured with custom settings.

var `decimal` → [Formatter.Decimal](#) *read-only*

Returns a new number formatter that will use both a decimal separator.

var `percent` → [Formatter.Decimal](#) *read-only*

Returns a new number formatter that will display the value as a percentage.

var `percentWithDecimal` → [Formatter.Decimal](#) *read-only*

Returns a new number formatter that will display the value as a percentage with a decimal separator.

var `plain` → [Formatter.Decimal](#) *read-only*

Returns a new number formatter that will not use any separators.

var `thousandsAndDecimal` → [Formatter.Decimal](#) *read-only*

Returns a new number formatter that will use both a thousands and decimal separator.

Instance Functions

function `stringFromDecimal`(number: [Decimal](#)) → String or null

Format a `Decimal` as a string, based on the rules set on the formatter.

function `decimalFromString`(string: String) → Decimal or null

Parses a `Decimal` from a string, based on the rules set on the formatter. Returns null if the value was not recognized.

Instance Properties

var `decimalSeparator` → String

The string to display between the whole portion of a number and the decimal portion.

var `negativeFormat` → String

A format string to use for negative values.

var `positiveFormat` → String

A format string to use for positive values.

var `thousandsSeparator` → String or null

The string to display between groups of digits representing powers of a thousand.

var `zeroSymbol` → String or null

The string to use when displaying a zero value. If this is null, the `positiveFormat` is used.

Formatter.Duration : Formatter

Constructors

`new Formatter.Duration()` → Formatter.Duration

Instance Functions

`function stringFromDecimal(number: Decimal)` → String or null

`function decimalFromString(string: String)` → Decimal or null

Instance Properties

`var hoursPerDay` → Number

`var hoursPerWeek` → Number

`var useVerboseFormat` → Boolean

Formatter.Date.Style

Class Properties

`var Full` → Formatter.Date.Style *read-only*

Use the user's “full” format as selected in system settings.

var **Long** → [Formatter.Date.Style](#) *read-only*

Use the user’s “long” format as selected in system settings.

var **Medium** → [Formatter.Date.Style](#) *read-only*

Use the user’s “medium” format as selected in system settings.

var **Short** → [Formatter.Date.Style](#) *read-only*

Use the user’s “short” format as selected in system settings.

var **all** → Array of [Formatter.Date.Style](#) *read-only*

Function

The built-in JavaScript [Function](#) constructor.

Image

Class Functions

function **symbolNamed**(name: String) → [Image](#) or null

Returns an image given a symbol name.

LigatureStyle

Class Properties

var `All` → [LigatureStyle](#) *read-only*

Use all of the available ligatures.

var `Essential` → [LigatureStyle](#) *read-only*

Use ligatures that are required for proper rendering of text.

var `Standard` → [LigatureStyle](#) *read-only*

Use the default ligatures for the given script.

var `all` → Array of [LigatureStyle](#) *read-only*

Locale

Class Properties

var `identifiers` → Array of String *read-only*

The list of known ISO locale identifiers.

Constructors

new `Locale`(identifier: String) → [Locale](#)

Instance Properties

var `calendar` → [Calendar](#) *read-only*

The calendar for the locale.

var `currencyCode` → String **or null** *read-only*

The currency code for the locale.

var `identifier` → String *read-only*

The ISO locale identifier for this object.

MenuItem

Instance Properties

var `checked` → Boolean

If true, a checkmark is displayed next to the MenuItem's label.

var `image` → [Image](#) **or null**

An optional image to be displayed with the MenuItem.

var `label` → String

The string displayed to describe the MenuItem's action.

NamedStyle.List

Instance Functions

function `add`(name: String or null) → [NamedStyle](#)

Makes a new `NamedStyle` at the end of the `NamedStyleList`, and optionally assigns it a name.

function `byName`(name: String) → [NamedStyle](#) or null

Returns the first named style that has the specified name, or null if none do.

function `byIdentifier`(identifier: String) → [NamedStyle](#) or null

Returns the single named style with the specified identifier, or null if no style has that identifier.

function `moveStyles`(styles: Array of [NamedStyle](#), position: [NamedStylePosition](#))

Reorders the named styles within the `NamedStyleList`. This cannot be used to move styles between documents.

function `duplicateStyles`(styles: Array of [NamedStyle](#), position: [NamedStylePosition](#)) → Array of [NamedStyle](#)

Instance Properties

var `all` → Array of [NamedStyle](#) *read-only*

Returns the list of all NamedStyles. Note that the order determine which attribute values are applied if two named styles have conflicting settings.

var `beginning` → [NamedStylePosition](#) *read-only*

Returns a NamedStylePosition that indicates the position before any existing named styles.

var `end` → [NamedStylePosition](#) *read-only*

Returns a NamedStylePosition that indicates the position before after existing named styles.

NamedStylePosition

Notification

Constructors

new `Notification`(title: String) → [Notification](#)

Instance Functions

function `show()` → [Promise](#) of [Notification](#)

Attempts to present the notification and returns a Promise which will yield the notification object itself if it is clicked or tapped, or an error if it cannot be presented

or is dismissed.

Instance Properties

var `subtitle` → String or null

var `title` → String

ObjectIdentifier

A unique identifier referring to a DatabaseObject.

Instance Properties

var `objectClass` → [Object](#) or null *read-only*

Returns the constructor object that would be used for instances of the class for this ObjectIdentifier.

var `primaryKey` → String *read-only*

Returns the primary key of the object identifier.

Pasteboard

A pasteboard temporarily holds representations of items of different types for transfer between different applications or different locations in the application.

Class Functions

function `makeUnique()` → [Pasteboard](#)

Creates a new unique pasteboard.

Class Properties

var `general` → [Pasteboard](#) *read-only*

The Pasteboard used for user-initiated copy/paste support.

Instance Functions

function `availableType(types: Array of TypeIdentifier)`
→ [TypeIdentifier](#) or null

The first type from the provided list which is available on the pasteboard, or null if none are available.

function `addItem(items: Array of Pasteboard.Item)`

Appends the new items to the pasteboard.

function `clear()`

Remove all items from the pasteboard.

function `dataForType(type: TypeIdentifier)` → [Data](#) or null

The Data representation for the given type in this pasteboard, or `null` if none is available.

```
function setDataForType(data: Data, type: TypeIdentifier)
```

Set the Data representation for the given type in this pasteboard, replacing any previously set data.

```
function stringForType(type: TypeIdentifier) → String or null
```

The String representation for the given type in this pasteboard, or `null` if none is available.

```
function setStringForType(string: String, type: TypeIdentifier)
```

Set the String representation for the given type in this pasteboard, replacing any previously set data.

Instance Properties

```
var URL → URL or null
```

Gets or sets the pasteboard content as a single URL.

```
var URLs → Array of URL or null
```

Gets or sets the pasteboard content as a list of URLs.

var `color` → [Color](#) or null

Gets or sets the pasteboard content as a single color.

var `colors` → Array of [Color](#) or null

Gets or sets the pasteboard content as a list of colors.

var `hasColors` → Boolean ***read-only***

Returns true if the pasteboard contains one or more colors.

var `hasImages` → Boolean ***read-only***

Returns true if the pasteboard contains one or more images.

var `hasStrings` → Boolean ***read-only***

Returns true if the pasteboard contains one or more strings.

var `hasURLs` → Boolean ***read-only***

Returns true if the pasteboard contains one or more URLs.

var `image` → [Image](#) or null

Gets or sets the pasteboard content as a single image.

var `images` → Array of [Image](#) or null

Gets or sets the pasteboard content as a list of images.

var `items` → Array of [Pasteboard.Item](#)

The array of individual items on the pasteboard, each potentially with their own set of types.

var `string` → String or null

Gets or sets the pasteboard content as a single plain-text string.

var `strings` → Array of String or null

Gets or sets the pasteboard content as a list of plain-text strings.

var `types` → Array of [TypeIdentifier](#) *read-only*

The list of pasteboard types currently available on the pasteboard.

Pasteboard.Item

Constructors

new [Pasteboard.Item\(\)](#) → [Pasteboard.Item](#)

Make a new empty pasteboard item with no contents.

Instance Functions

```
function dataForType(type: TypeIdentifier) → Data or  
null
```

The `Data` representation for the given type in this pasteboard item, or `null` if none is available.

```
function setDataForType(data: Data, type:  
TypeIdentifier)
```

Set the `Data` representation for the given type in this pasteboard item, replacing any previously set data.

```
function stringForType(type: TypeIdentifier) → String or  
null
```

The `String` representation for the given type in this pasteboard item, or `null` if none is available.

```
function setStringForType(string: String, type:  
TypeIdentifier)
```

Set the `String` representation for the given type in this pasteboard item, replacing any previously set data.

Instance Properties

```
var types → Array of TypeIdentifier read-only
```

The list of types available for this pasteboard item.

Perspective

Class Properties

var `all` → Array of [Perspective.BuiltIn](#) or [Perspective.Custom](#) *read-only*

Returns all the built-in and custom perspectives.

Perspective.BuiltIn

Class Properties

var `Flagged` → [Perspective.BuiltIn](#) *read-only*

The flagged items.

var `Forecast` → [Perspective.BuiltIn](#) *read-only*

The upcoming due items.

var `Inbox` → [Perspective.BuiltIn](#) *read-only*

The inbox of tasks.

var `Nearby` → [Perspective.BuiltIn](#) *read-only*

Nearby items on a map (iOS only).

var **Projects** → Perspective.BuiltIn *read-only*

The library of projects.

var **Review** → Perspective.BuiltIn *read-only*

The projects needing review.

var **Search** → Perspective.BuiltIn *read-only*

A search of the database. This perspective cannot be set, but might be reported if the user is searching.

var **Tags** → Perspective.BuiltIn *read-only*

The hierarchy of tags.

var **all** → Array of Perspective.BuiltIn *read-only*

Instance Properties

var **name** → String *read-only*

The name of the built in perspective.

PlugIn

Class Functions

function `find`(identifier: String, minimumVersion: [Version](#) or null) → [PlugIn](#) or null

Class Properties

var `all` → Array of [PlugIn](#) *read-only*

Instance Functions

function `library`(identifier: String) → [PlugIn.Library](#) or null

Looks for a `PlugIn.Library` in the receiver and returns it if found.

function `action`(identifier: String) → [PlugIn.Action](#) or null

function `handler`(identifier: String) → [PlugIn.Handler](#) or null

function `resourceNamed`(name: String) → [URL](#) or null

function `imageNamed`(name: String) → [Image](#) or null

function `localizedResourceNamed`(filename: String) → [FileWrapper](#) or null

Instance Properties

var `URL` → [URL](#) or null *read-only*

Returns the original URL from whence this PlugIn came, if known.

var `actions` → Array of [PlugIn.Action](#) *read-only*

var `author` → String *read-only*

Returns the author for the PlugIn.

var `description` → String *read-only*

Returns the description provided for the PlugIn.

var `displayName` → String *read-only*

Returns the localized, human-readable name for the PlugIn.

var `handlers` → Array of [PlugIn.Handler](#) *read-only*

var `identifier` → String *read-only*

The unique identifier of the PlugIn.

var `libraries` → Array of [PlugIn.Library](#) *read-only*

var `version` → [Version](#) *read-only*

Returns the current Version for the PlugIn.

PlugIn.Action

Constructors

new `PlugIn.Action`(perform: [Function](#)) → [PlugIn.Action](#)

Returns a new `PlugIn.Action`. Only used within an action JavaScript file embedded within a `PlugIn`.

Instance Properties

var `description` → String *read-only*

var `label` → String *read-only*

Returns the default label to use for interface controls that invoke the action.

var `longLabel` → String *read-only*

Returns the label to use for interface controls that invoke the action, when a long amount of space is available.

var `mediumLabel` → String *read-only*

Returns the label to use for interface controls that invoke the action, when a medium amount of space is available.

var `name` → String *read-only*

Returns the name of the `PlugIn.Action`.

var `paletteLabel` → String *read-only*

Returns the label to use for interface controls that show a prototype of the action control, such as on a macOS toolbar configuration sheet.

var `perform` → [Function](#) *read-only*

var `plugIn` → [PlugIn](#) *read-only*

Returns the `PlugIn` that contains this object.

var `shortLabel` → String *read-only*

Returns the label to use for interface controls that invoke the action, when a short amount of space is available.

var `validate` → [Function](#) or null

A function to check whether the action is supported, given the current application state, as determined by the arguments passed (typically including the selection). This optional Function may be configured while the `Action` is being loaded, but after that the `Action` will be frozen.

PlugIn.Handler

Constructors

new `PlugIn.Handler`(`invoke`: [Function](#)) → [PlugIn.Handler](#)

Returns a new `PlugIn.Handler`. Only used within an handler JavaScript file embedded within a `PlugIn`.

Instance Properties

var `invoke` → [Function](#) *read-only*

The Function that will be executed for each handler registered for an event posted by an application object.

var `name` → String *read-only*

Returns the name of the `PlugIn.Handler`.

var `plugIn` → [PlugIn](#) *read-only*

Returns the `PlugIn` that contains this object.

var `willAttach` → [Function](#) or null

An optional Function that can be set on `PlugIn.Handler` as it is being loaded (but not after). This function is passed the application object that post events to trigger the handler. The return value should be a state object that is JSON archivable (or undefined if the handler has no state to maintain across invocations).

var `willDetach` → [Function](#) or null

An optional Function that can be set on `PlugIn.Handler` as it is being loaded (but not after). Called when a previously attached `PlugIn.Handler` is being detached from an application object. Any return value or thrown error are ignored.

PlugIn.Library

An object that represents a library from a plug-in.

Constructors

new `PlugIn.Library`(version: [Version](#)) → [PlugIn.Library](#)

Returns a new `Library`. Typically only used within a library JavaScript file embedded within a `PlugIn`.

Instance Properties

var `name` → String *read-only*

Returns the name of the `PlugIn.Library`.

var `plugIn` → [PlugIn](#) *read-only*

Returns the `PlugIn` that contains this object.

var `version` → [Version](#) *read-only*

Returns the `Version` of this library, as passed to the constructor.

Preferences

Constructors

new `Preferences`(identifier: String or null) → [Preferences](#)

Creates a new Preferences instance. The identifier specified may be `null` to create an instance for the currently loading plug-in. If it is `null` and a plug-in is not being loaded, an error will be thrown. Key/value pairs stored in the instance will be prefixed with the identifier and a “.”.

Instance Functions

function `read(key: String) → Object or null`

Returns the previously stored value for the given key, or `null` if no value is stored.

function `readBoolean(key: String) → Boolean`

Returns the previously stored value as a `Boolean`, or `false` if there is no stored value or it can't be converted to a `Boolean`.

function `readString(key: String) → String or null`

Returns the previously stored value as a `String`, or `null` if there is no stored value or it is not a `String`.

function `readNumber(key: String) → Number`

Returns the previously stored value as a `Number`, or `null` if there is no stored value or it is not a `Number`.

function `readDate(key: String) → Date or null`

Returns the previously stored value as a `Date`, or `null` if there is no stored value or it is not a `Date`.

function `readData`(key: String) → [Data](#) or null

Returns the previously stored value as a `Data`, or null if there is no stored value or it is not a `Data`.

function `write`(key: String, value: Boolean, String, Number, Date, or [Data](#) or null)

Stores the specified key/value pair, or removes the pair if value is null.

function `remove`(key: String)

Removes and previously stored value for the given key.

Instance Properties

var `identifier` → String *read-only*

The scoping identifier the instance given when created, or the plug-in identifier if none was given.

Project.ReviewInterval

`Project.ReviewInterval` is a value object which represents a simple repetition interval. Because it's a value object rather than a proxy, changing its properties doesn't affect any projects directly. To change a project's review interval, update the value and assign it back to the project's `reviewInterval` property:

```
let project = projectNamed("Miscellaneous");
let reviewInterval = project.reviewInterval;
reviewInterval.steps = 3;
```

```
reviewInterval.unit = "months";  
project.reviewInterval = reviewInterval;
```

Note: At one time these simple repetition intervals were also used for task repetitions, but over time we replaced those with the more flexible `Task.RepetitionRule`. Eventually we expect to also replace this review interval with flexible repetition rules.

Instance Properties

var `steps` → Number

The count of units to use for this interval (e.g. “14” days or “12” months).

var `unit` → String

The units to use (e.g. “days”, “weeks”, “months”, “years”).

Project.Status

Class Properties

var `Active` → [Project.Status](#) *read-only*

The project is active.

var `Done` → [Project.Status](#) *read-only*

The project has been marked as completed.

var **Dropped** → [Project.Status](#) *read-only*

The project has been dropped.

var **OnHold** → [Project.Status](#) *read-only*

The project has been put on-hold.

var **all** → Array of [Project.Status](#) *read-only*

Promise

The built-in JavaScript [Promise](#) constructor.

Selection

An object representing the current selection in a Window.

Instance Properties

var **allObjects** → Array of [Object](#) *read-only*

Returns all the objects in the selection.

var **database** → [Database](#) or null *read-only*

Returns the Database object in the selection, if any.

var `databaseObjects` → Array of [DatabaseObject](#) *read-only*

Returns all the DatabaseObject objects in the selection, if any.

var `document` → [DatabaseDocument](#) or null *read-only*

The Document containing the selection.

var `folders` → [FolderArray](#) *read-only*

Returns all the Folder objects in the selection, if any.

var `projects` → [ProjectArray](#) *read-only*

Returns all the Project objects in the selection, if any.

var `tags` → [TagArray](#) *read-only*

Returns all the Tag objects in the selection, if any.

var `tasks` → [TaskArray](#) *read-only*

Returns all the Task objects in the selection, if any.

var `window` → [DocumentWindow](#) or null *read-only*

The Window containing the selection.

Settings

Settings represent the database synchronized configuration values. *NOTE:* editing these should be done with care, as storing invalid values may corrupt your database or produce instability in the various client applications.

Instance Functions

function `defaultObjectForKey(key: String)` → [`Object`](#) or null

function `hasNonDefaultObjectForKey(key: String)` → Boolean

function `objectForKey(key: String)` → [`Object`](#) or null

function `setObjectForKey(value: Object or null, key: String)`

function `boolForKey(key: String)` → Boolean

function `setBoolForKey(value: Boolean, key: String)`

function `integerForKey(key: String)` → Number

function `setIntegerForKey(value: Number, key: String)`

Instance Properties

var `keys` → Array of String *read-only*

SharePanel

An interface that can display the system share interaction for the given items.

Constructors

new `SharePanel`(items: Array of [URL](#), String, [Image](#), or [FileWrapper](#)) → [SharePanel](#)

Create a new share panel with the given items.

Instance Functions

function `addItem`(shareItem: [URL](#), String, [Image](#), or [FileWrapper](#))

Appends the item to the end of items.

function `addItems`(shareItems: Array of [URL](#), String, [Image](#), or [FileWrapper](#))

Appends the contents of the given array to the end of items.

function `removeItem`(shareItem: [URL](#), String, [Image](#), or [FileWrapper](#))

Removes the first occurrence of the item from items if it is present in items.

function `removeItems`(shareItems: Array of [URL](#), String, [Image](#), or [FileWrapper](#))

Removes the first occurrence of each member of the given array from `items` if that member is present in `items`.

```
function clearItems()
```

Sets `items` to an empty array. Note: Calling `show` when `items` is empty results in an error, so be sure to add new items after calling this and before calling `show`.

```
function show()
```

Presents the share panel for its items. Calling this when `items` is empty will result in an error.

Instance Properties

var `items` → Array of [URL](#), String, [Image](#), or [FileWrapper](#)

The items that will be supplied to the system share interaction upon calling `show`.

Speech

Speech.Boundary

Class Properties

var `Immediate` → [Speech.Boundary](#) *read-only*

var `Word` → [Speech.Boundary](#) *read-only*

var `all` → Array of [Speech.Boundary](#) *read-only*

Speech.Synthesizer

Constructors

new `Speech.Synthesizer()` → [Speech.Synthesizer](#)

Instance Functions

function `speakUtterance`(utterance: [Speech.Utterance](#))

Enqueues the utterance. If the utterance is already or enqueued or speaking, throws and error.

function `stopSpeaking`(boundary: [Speech.Boundary](#)) → Boolean

function `pauseSpeaking`(boundary: [Speech.Boundary](#)) → Boolean

function `continueSpeaking`() → Boolean

Instance Properties

var `paused` → Boolean *read-only*

var `speaking` → Boolean *read-only*

Speech.Utterance

Class Properties

var `defaultSpeechRate` → Number *read-only*

var `maximumSpeechRate` → Number *read-only*

var `minimumSpeechRate` → Number *read-only*

Constructors

new `Speech.Utterance`(string: String) → [`Speech.Utterance`](#)

Instance Properties

var `pitchMultiplier` → Number

A value between 0.5 and 2.0, controlling the pitch of the utterance.

var `postUtteranceDelay` → Number

var `preUtteranceDelay` → Number

var `prefersAssistiveTechnologySettings` → Boolean

If an assistive technology is on, like VoiceOver, the user's selected voice, rate and other settings will be used for this speech utterance instead of the default values. If no assistive technologies are on, then the values of the properties on `AVSpeechUtterance` will be used. Note that querying the properties will not reflect the user's settings.

var `rate` → Number

A value between `Speech.Utterance.minimumSpeechRate` and `Speech.Utterance.maximumSpeechRate` controlling the rate of speech for the utterance.

var `string` → `String` **or null** *read-only*

var `voice` → [`Speech.Voice`](#) **or null**

The voice to use for this utterance, or `null` in which case the default voice will be used.

var `volume` → `Number`

A value between 0.0 and 1.0 controller the volume of the utterance.

Speech.Voice

Class Functions

function `withLanguage`(code: `String` **or null**) → [`Speech.Voice`](#) **or null**

Returns a voice for the given BCP-47 language code (such as `en-US` or `fr-CA`), or the default voice if passed `null`. Returns `null` for an invalid language code.

function `withIdentifier`(identifier: `String`) → [`Speech.Voice`](#) **or null**

Returns the voice with the given identifier, or `null` if not found.

Class Properties

var `allVoices` → Array of [Speech.Voice](#) *read-only*

var `currentLanguageCode` → String *read-only*

Instance Properties

var `gender` → [Speech.Voice.Gender](#) *read-only*

var `identifier` → String *read-only*

var `language` → String *read-only*

var `name` → String *read-only*

Speech.Voice.Gender

Class Properties

var `Female` → [Speech.Voice.Gender](#) *read-only*

var `Male` → [Speech.Voice.Gender](#) *read-only*

var `Unspecified` → [Speech.Voice.Gender](#) *read-only*

var `all` → Array of [Speech.Voice.Gender](#) *read-only*

StringEncoding

Class Properties

var ASCII → [StringEncoding](#) *read-only*

var ISO2022JP → [StringEncoding](#) *read-only*

var ISOLatin1 → [StringEncoding](#) *read-only*

var ISOLatin2 → [StringEncoding](#) *read-only*

var JapaneseEUC → [StringEncoding](#) *read-only*

var MacOSRoman → [StringEncoding](#) *read-only*

var NextStep → [StringEncoding](#) *read-only*

var NonLossyASCII → [StringEncoding](#) *read-only*

var ShiftJIS → [StringEncoding](#) *read-only*

var Symbol → [StringEncoding](#) *read-only*

var UTF16 → [StringEncoding](#) *read-only*

var UTF16BigEndian → [StringEncoding](#) *read-only*

var UTF16LittleEndian → [StringEncoding](#) *read-only*

var UTF32 → [StringEncoding](#) *read-only*

var UTF32BigEndian → [StringEncoding](#) *read-only*

var UTF32LittleEndian → [StringEncoding](#) *read-only*

var UTF8 → [StringEncoding](#) *read-only*

var Unicode → [StringEncoding](#) *read-only*

var `WindowsCP1250` → [StringEncoding](#) *read-only*

var `WindowsCP1251` → [StringEncoding](#) *read-only*

var `WindowsCP1252` → [StringEncoding](#) *read-only*

var `WindowsCP1253` → [StringEncoding](#) *read-only*

var `WindowsCP1254` → [StringEncoding](#) *read-only*

var `all` → Array of [StringEncoding](#) *read-only*

Style

Instance Functions

function `set`(attribute: [Style.Attribute](#), value: [Object](#) or null) → Boolean

Sets (or clears) the value for the given style attribute. Styles that cascade from this one will inherit this value, if they don't define their own value or have a closer ancestor style that does. Returns true if a change was actually made, false otherwise. Throws an error if the key does not map to a known attribute, or if the value is of the wrong type for the specified attribute.

function `get`(attribute: [Style.Attribute](#)) → [Object](#) or null

Looks up the value for the specified style attribute locally, in the cascading and inherited styles, and finally falling back to the default value for the style attribute.

function `localValueForAttribute(attribute: Style.Attribute)` → [Object](#) or null

Looks up the value for the specified style attribute locally, returning null if it is not set.

function `addNamedStyle(namedStyle: NamedStyle)`

Adds the specified `NamedStyle` to the set of named styles to include in this `Style`. If the style is already present, or if this would create a loop (adding two `NamedStyles` to each other's list of named styles), an error will be thrown.

function `removeNamedStyle(namedStyle: NamedStyle)`

Removes the specified `NamedStyle` from the set of named styles to include in this `Style`. If the style is not present, an error will be thrown.

function `influencedBy(otherStyle: Style)` → Boolean

Returns true if the receiver is influenced, directly or indirectly from the passed `Style`.

function `setStyle(style: Style)`

Updates all the attributes and inherited styles on the receiver to be the same as the argument `Style`.

function `clear()`

Removes all the locally applied style attribute values for this `Style`.

Instance Properties

var `fontFillColor` → [Color](#)

The color used to fill text. Setting the color to `null` will remove the setting for this style.

var `link` → [URL](#) or `null` ***read-only***

Returns the URL link for a style, or `null` if there is no link applied. Note that `get(Style.Attribute.Link)` on the same style will return the default URL with an empty `toString()` value when there is no URL applied. If the style represents a file attachment and there is no specific link attribute set, the URL for the file attachment will be returned. If the style represents an file attachment that is embedded in the document, `null` will be returned.

var `locallyDefinedAttributes` → Array of [Style.Attribute](#) ***read-only***

Returns an array of the `Style.Attributes` defined on this `Style`.

var `namedStyles` → Array of [NamedStyle](#) ***read-only***

Returns the `NamedStyles` that are directly associated with this `Style`. If a style attribute lookup doesn't find a value in the local style, then the named styles will be searched.

NamedStyle : [Style](#)

Instance Functions

```
function remove()
```

Removes the NamedStyle from the document. Any references to it are disconnected as well.

Instance Properties

```
var after → NamedStylePosition read-only
```

Returns a NamedStylePosition that indicates the slot after this item.

```
var before → NamedStylePosition read-only
```

Returns a NamedStylePosition that indicates the slot before this item.

```
var identifier → String read-only
```

A unique identifier for the style, which is suitable for long-lived references.

```
var name → String
```

The name of the style that is presented in the interface.

Style.Attribute

Class Properties

```
var BackgroundColor → Style.Attribute read-only
```

var BaselineOffset → [Style.Attribute](#) *read-only*

var BaselineSuperscript → [Style.Attribute](#) *read-only*

var Expansion → [Style.Attribute](#) *read-only*

var FontCondensed → [Style.Attribute](#) *read-only*

var FontFamily → [Style.Attribute](#) *read-only*

var FontFillColor → [Style.Attribute](#) *read-only*

var FontFixedPitch → [Style.Attribute](#) *read-only*

var FontItalic → [Style.Attribute](#) *read-only*

var FontName → [Style.Attribute](#) *read-only*

var FontNarrow → [Style.Attribute](#) *read-only*

var FontSize → [Style.Attribute](#) *read-only*

var FontStrokeColor → [Style.Attribute](#) *read-only*

var FontStrokeWidth → [Style.Attribute](#) *read-only*

var FontWeight → [Style.Attribute](#) *read-only*

var KerningAdjustment → [Style.Attribute](#) *read-only*

var LigatureSelection → [Style.Attribute](#) *read-only*

var Link → [Style.Attribute](#) *read-only*

var Obliqueness → [Style.Attribute](#) *read-only*

var ParagraphAlignment → [Style.Attribute](#) **read-only**

var ParagraphBaseWritingDirection → [Style.Attribute](#) **read-only**

var ParagraphDefaultTabInterval → [Style.Attribute](#) **read-only**

var ParagraphFirstLineHeadIndent → [Style.Attribute](#) **read-only**

var ParagraphHeadIndent → [Style.Attribute](#) **read-only**

var ParagraphLineHeightMultiple → [Style.Attribute](#) **read-only**

var ParagraphLineSpacing → [Style.Attribute](#) **read-only**

var ParagraphMaximumLineHeight → [Style.Attribute](#) **read-only**

var ParagraphMinimumLineHeight → [Style.Attribute](#) **read-only**

var ParagraphSpacing → [Style.Attribute](#) **read-only**

var ParagraphSpacingBefore → [Style.Attribute](#) **read-only**

var ParagraphTabStops → [Style.Attribute](#) **read-only**

var ParagraphTailIndent → [Style.Attribute](#) **read-only**

var ShadowBlurRadius → [Style.Attribute](#) **read-only**

`var ShadowColor → Style.Attribute read-only`

`var ShadowOffset → Style.Attribute read-only`

`var StrikethroughAffinity → Style.Attribute read-only`

`var StrikethroughColor → Style.Attribute read-only`

`var StrikethroughPattern → Style.Attribute read-only`

`var StrikethroughStyle → Style.Attribute read-only`

`var UnderlineAffinity → Style.Attribute read-only`

`var UnderlineColor → Style.Attribute read-only`

`var UnderlinePattern → Style.Attribute read-only`

`var UnderlineStyle → Style.Attribute read-only`

Instance Properties

`var defaultValue → Object read-only`

Returns the default value that will be used when a style has no local value for this attribute, nor do any of its associated styles.

`var key → String read-only`

Returns the string used to identify this attribute when calling `get` or `set` on a `Style` instance.

Tag.ChildInsertionLocation

A location specified relative to an existing Tag or Database. These cannot be instantiated directly, rather they are returned from properties like `Tag.before` or `Database.beginning`.

Tag.Status

Class Properties

var `Active` → Tag.Status *read-only*

The tag is active.

var `Dropped` → Tag.Status *read-only*

The tag has been dropped.

var `OnHold` → Tag.Status *read-only*

The tag has been put on-hold.

var `all` → Array of Tag.Status *read-only*

Task.ChildInsertionLocation

A location specified relative to an existing Task or Database. These cannot be instantiated directly, rather they are returned from properties like `Task.before`, `Inbox.ending`, or `Project.beginning`. (For a complete list of locations, open

the navigation sidebar and use its filter to search for Task.ChildInsertionLocation.)

Task.Notification.Kind

Class Properties

var **Absolute** → [Task.Notification.Kind](#) *read-only*

This notification fires on a given date, regardless of its task's due and defer dates.

var **DueRelative** → [Task.Notification.Kind](#) *read-only*

This notification fires at a time relative to its task's due date.

var **Unknown** → [Task.Notification.Kind](#) *read-only*

It is not known what this notification's fire date is relative to.

var **all** → Array of [Task.Notification.Kind](#) *read-only*

Task.RepetitionMethod

Class Properties

var **DeferUntilDate** → [Task.RepetitionMethod](#) *read-only*

var **DueDate** → [Task.RepetitionMethod](#) *read-only*

var `Fixed` → [Task.RepetitionMethod](#) *read-only*

var `None` → [Task.RepetitionMethod](#) *read-only*

The task does not repeat.

var `all` → Array of [Task.RepetitionMethod](#) *read-only*

Task.RepetitionRule

A `Task.RepetitionRule` describes a pattern of dates using a ICS formatted recurrence string and a `Task.RepetitionMethod` to describe how those dates are applied to a Task.

Constructors

new `Task.RepetitionRule`(ruleString: String, method: [Task.RepetitionMethod](#)) → [Task.RepetitionRule](#)

Returns a new `Task.RepetitionRule` with the specified ICS rule string and repetition method. If the rule string is not valid, or combination with the method is not supported, an error will be thrown.

Instance Functions

function `firstDateAfterDate`(date: Date) → Date

Returns the first date described by the repetition rule that is after the given date.

Instance Properties

var `method` → [Task.RepetitionMethod](#) *read-only*

The method used to create the repetition rule.

var `ruleString` → String *read-only*

The ICS rule string used to create the repetition rule.

Task.Status

Class Properties

var `Available` → [Task.Status](#) *read-only*

The task is available to work on.

var `Blocked` → [Task.Status](#) *read-only*

The task is not available to work on currently, due to a future defer date, a preceeding task in a sequential project, or having an on-hold tag associated.

var `Completed` → [Task.Status](#) *read-only*

The task is already completed.

var `Dropped` → [Task.Status](#) *read-only*

The task will not be worked on.

var **DueSoon** → Task.Status *read-only*

The task is incomplete and due soon.

var **Next** → Task.Status *read-only*

The task is the first available task in a project.

var **Overdue** → Task.Status *read-only*

The task is incomplete overdue.

var **all** → Array of Task.Status *read-only*

Text

Class Functions

function **makeFileAttachment**(fileWrapper: FileWrapper, style: Style) → Text

Returns a new Text instance that represents a file attachment. The attachment has a single character string content with a special value.

Constructors

new **Text**(string: String, style: Style) → Text

Returns a new Text instance with the given string contents and Style applied to the entire range of text.

Instance Functions

function `textInRange`(range: [Text.Range](#)) → [Text](#)

Returns a copy of the text in the specified range.

function `styleForRange`(range: [Text.Range](#)) → [Style](#)

Returns a Style instance for the given range of the Text.

function `ranges`(component: [TextComponent](#),
useEnclosingRange: Boolean or null) → Array of
[Text.Range](#)

Returns an array of TextRanges for the specified component. If useEnclosingRange is true, then any extra characters that separate follow a component will be included in its range. Any extra characters before the first found component will be included in the first range.

function `replace`(range: [Text.Range](#), with: [Text](#))

Replaces the sub-range of the receiving Text with a copy of the passed in Text (which remains unchanged).

function `append`(text: [Text](#))

Appends the given Text to the receiver.

```
function insert(position: Text.Position, text: Text)
```

Inserts a copy of the given Text at the specified position in the receiver.

```
function remove(range: Text.Range)
```

Removes the indicated sub-range of the receiving Text.

```
function find(string: String, options: Array of  
Text.FindOption or null, range: Text.Range or null) →  
Text.Range or null
```

Finds an occurrence of `string` within the Text and returns the enclosing Text.Range if there is a match. If range is passed, only that portion of the Text is searched. The supplied options, if any, change how the search is performed based on their definition.

Instance Properties

```
var attachments → Array of Text read-only
```

Returns an array of copies of the blocks of Text in the receiver that represent Attachments. Note that editing these instances will not change the original.

```
var attributeRuns → Array of Text read-only
```

Returns an array of copies of the contiguous blocks of Text in the receiver that have the same style. Note that editing these instances will not change the original.

var **characters** → Array of Text *read-only*

Returns an array of copies of the characters in the Text. Note that editing these instances will not change the original.

var **end** → Text.Position *read-only*

Returns a Text.Position indicating the end of the Text.

var **fileWrapper** → FileWrapper or null *read-only*

Returns the attached file wrapper for the Text (or rather, the first character of the text), if any.

var **paragraphs** → Array of Text *read-only*

Returns an array of copies of the paragraphs in the Text. Note that editing these instances will not change the original. Paragraphs, if ended by a newline, will contain the newline character.

var **range** → Text.Range *read-only*

Returns a Text.Range that spans the entire Text.

var **sentences** → Array of Text *read-only*

Returns an array of copies of the sentences in the Text. Note that editing these instances will not change the original.

var **start** → [Text.Position](#) *read-only*

Returns a `Text.Position` indicating the beginning of the `Text`.

var **string** → `String`

Returns a plain `String` version of the characters in the `Text`. Note that since JavaScript represents Strings as Unicode code points, the length of the returned string may be different from the number of characters in the `Text` object.

var **style** → [Style](#) *read-only*

Returns a `Style` instance for this `Text` object.

var **words** → `Array` of [Text](#) *read-only*

Returns an array of copies of the words in the `Text`. Note that editing these instances will not change the original.

Text.FindOption

Class Properties

var **Anchored** → [Text.FindOption](#) *read-only*

Matches must be anchored to the beginning (or end if `Backwards` is include) of the string or search range.

var **Backwards** → [Text.FindOption](#) *read-only*

Search starting from the end of the string or range.

var `CaseInsensitive` → [`Text.FindOption`](#) *read-only*

Compare upper and lower case characters as equal.

var `DiacriticInsensitive` → [`Text.FindOption`](#) *read-only*

Ignore diacritics. For example, “ö” is considered the same as “o”.

var `ForcedOrdering` → [`Text.FindOption`](#) *read-only*

Force an ordering between strings that aren’t strictly equal.

var `Literal` → [`Text.FindOption`](#) *read-only*

Perform exact character-by-character matching.

var `Numeric` → [`Text.FindOption`](#) *read-only*

Order numbers by numeric value, not lexicographically. Only applies to ordered comparisons, not find operations.

var `RegularExpression` → [`Text.FindOption`](#) *read-only*

For find operations, the string is treated as an ICU-compatible regular expression. If set, no other options can be used except for `CaseInsensitive` and `Anchored`.

var `WidthInsensitive` → [`Text.FindOption`](#) *read-only*

Ignore width differences. For example, “a” is considered the same as ‘FULLWIDTH LATIN SMALL LETTER A’ (U+FF41).

var `all` → Array of [Text.FindOption](#) *read-only*

Text.Position

Text.Range

Constructors

new `Text.Range`(start: [Text.Position](#), end: [Text.Position](#)) → [Text.Range](#)

Instance Properties

var `end` → [Text.Position](#) *read-only*

Returns the `Text.Position` for the end of the `Text.Range`

var `isEmpty` → Boolean *read-only*

Returns true if the `Text.Range` contains no characters.

var `start` → [Text.Position](#) *read-only*

Returns the `Text.Position` for the beginning of the `Text.Range`

TextAlignment

Class Properties

var `Center` → [`TextAlignment`](#) *read-only*

Visually center aligned.

var `Justified` → [`TextAlignment`](#) *read-only*

Fully-justified.

var `Left` → [`TextAlignment`](#) *read-only*

Visually left aligned.

var `Natural` → [`TextAlignment`](#) *read-only*

Use the default alignment based on the characters in the text.

var `Right` → [`TextAlignment`](#) *read-only*

Visually right aligned.

var `all` → Array of [`TextAlignment`](#) *read-only*

TextComponent

Class Properties

var **Attachments** → TextComponent *read-only*

The ranges of Text which represent Attachments.

var **AttributeRuns** → TextComponent *read-only*

The ranges of Text which have the same attributes.

var **Characters** → TextComponent *read-only*

The individual characters of the Text. Note that some characters (like emoji) consist of multiple Unicode code points.

var **Paragraphs** → TextComponent *read-only*

The paragraphs of Text. Unlike other options, the line breaking characters that end the paragraph are included.

var **Sentences** → TextComponent *read-only*

The sentences of the Text.

var **Words** → TextComponent *read-only*

The words in the Text. Whitespace or other word break characters are not included.

var **all** → Array of TextComponent *read-only*

TimeZone

Class Properties

var `abbreviations` → Array of String *read-only*

The list of known time zone abbreviations.

Constructors

new `TimeZone`(abbreviation: String) → [TimeZone](#) or null

Make a new `TimeZone` with the given abbreviation. Note that the returned `TimeZone` may have a different abbreviation than the passed argument. For example, if one of “PST” or “PDT” is requested that doesn’t match the current use of daylight savings time, the one that does match will be returned.

Instance Properties

var `abbreviation` → String or null *read-only*

The abbreviation for the `TimeZone`.

var `daylightSavingTime` → Boolean *read-only*

Returns true if the `TimeZone` is currently using daylight savings time.

var `secondsFromGMT` → Number *read-only*

The current difference in seconds between this `TimeZone` and GMT.

Timer

Class Functions

```
function once(interval: Number, action:  
Function(timer: Timer)) → Timer
```

Makes a new `Timer` that will fire once, after the specified interval (in seconds from the current time). When the `Timer` fires, the passed in `Function` is called, passing the `Timer` as its argument.

```
function repeating(interval: Number, action:  
Function(timer: Timer)) → Timer
```

Makes a new `Timer` that will fire repeatedly with the specified interval (in seconds, with the first invocation happening that interval after the current time). When the `Timer` fires, the passed in `Function` is called, passing the `Timer` as its argument.

Instance Functions

```
function cancel()
```

Instance Properties

```
var interval → Number read-only
```

ToolbarItem

Instance Properties

var `image` → [Image](#) or null

var `label` → String

var `toolTip` → String or null

Tree

Instance Functions

function `nodeForObject`(object: [Object](#)) → [TreeNode](#) or null

Returns the `TreeNode` that represents the object in this `Tree`, or null if it cannot be found (possibly filtered out).

function `nodesForObjects`(object: Array of [Object](#)) → Array of [TreeNode](#)

Returns an array of `TreeNodes` for the objects that are currently in the `Tree`, according to the same filters as `nodeForObject()`. The size of the resulting node array may be smaller (even empty) than the passed in `objects` array.

function `reveal`(nodes: Array of [TreeNode](#))

Ensures the ancestor nodes of all the specified nodes are expanded.

function `select`(nodes: Array of [TreeNode](#), extending: Boolean or null)

Selects the specified `TreeNode`s that are visible (nodes with collapsed ancestors cannot be selected). If `extending` is `true`, the existing selection is not cleared.

```
function copyNodes(nodes: Array of TreeNode, to: Pasteboard)
```

Writes a serialized version of the nodes to the specified pasteboard.

```
function paste(from: Pasteboard, parentNode: TreeNode or null, childIndex: Number or null)
```

Attempts to read a serialized version of nodes from the pasteboard and create new items at the specified location in the receiver. If a parent node is not specified, then the root node of the receiver is assumed. If a `childIndex` is not specified, any new children are placed at the end of the parent's existing children.

Instance Properties

```
var rootNode → TreeNode read-only
```

Returns the `rootNode` of the Editor.

```
var selectedNodes → Array of TreeNode read-only
```

Returns the list of selected `TreeNode`s, in the order they appear in the tree.

ContentTree : [Tree](#)

SidebarTree : Tree

TreeNode

Instance Functions

function `childAtIndex`(childIndex: Number) → TreeNode

Returns the child node at the given index.

function `expand`(completely: Boolean or null)

Attempts to expand the `TreeNode`. If `completely` is passed, all the child nodes will be expanded as they allow.

function `collapse`(completely: Boolean or null)

Attempts to collapse the `TreeNode`. If `completely` is passed, all the child nodes will be collapse as they allow.

function `expandNote`(completely: Boolean or null)

Attempts to expand the inline note of the `TreeNode`. If `completely` is passed, all the child node notes will be expanded.

function `collapseNote`(completely: Boolean or null)

Attempts to collapse the inline note of the `TreeNode`. If `completely` is passed, all the child node notes will be collapsed.

```
function reveal()
```

Expands all the

```
function apply(function: Function(node: TreeNode) →  
omnjs AnonymousProxy or null) →  
omnjs AnonymousProxy or null
```

Calls the supplied function for each `TreeNode` in the receiver (including the receiver), passing that node as the single argument. The supplied function can optionally return a `ApplyResult` to skip enumeration of some elements.

Instance Properties

```
var canCollapse → Boolean read-only
```

Returns true if this `TreeNode` can be collapsed.

```
var canExpand → Boolean read-only
```

Returns true if this `TreeNode` can be expanded.

```
var childCount → Number read-only
```

Returns the number of children directly under this node.

```
var children → Array of TreeNode read-only
```

Returns the array of children that are visible under this node, according to any filtering that is being done, and in the order specified by any sorting rules that have

been established.

var `index` → Number ***read-only***

Returns the index of this `TreeNode` among its siblings, or zero for the `rootNode`.

var `isExpanded` → Boolean ***read-only***

Returns `true` if this `TreeNode` is currently expanded.

var `isNoteExpanded` → Boolean ***read-only***

Returns `true` if the note of this `TreeNode` is currently expanded.

var `isRevealed` → Boolean ***read-only***

Returns `true` if the `TreeNode` is the `rootNode` or all of its ancestor nodes are expanded.

var `isRootNode` → Boolean ***read-only***

Returns `true` if this node is the `rootNode` of its tree.

var `isSelectable` → Boolean ***read-only***

Returns `true` if this `TreeNode` can be selected. The `rootNode` cannot be selected, nor can nodes that aren't revealed.

var `isSelected` → Boolean

Set to `true` if this `TreeNode` is in the list of selected nodes for its tree. Attempting to set this to `true` will do nothing if the node is not revealed (or is the root node).

var `level` → `Number` *read-only*

Returns the nesting level of the `TreeNode`, relative to the root of the tree. The `rootNode` of an `Outline` has level zero, its children have level one, and so on. Note that if only a portion of the model is being shown, this level may not match the level of the underlying object.

var `object` → `Object` *read-only*

The model object which this node wraps.

var `parent` → `TreeNode` or `null` *read-only*

Returns the `TreeNode` that contains this node, or `null` if this is the `rootNode`.

var `rootNode` → `TreeNode` *read-only*

Returns the root `TreeNode` for the tree that this node belongs to.

TypeIdentifier

Class Functions

function `fromPathExtension`(`pathExtension`: `String`,
`isDirectory`: `Boolean`) → `TypeIdentifier`

Return a `TypeIdIdentifier` that matches items that have the given path extension and are (or are not) directories.

Class Properties

var `URL` → `TypeIdIdentifier` *read-only*

The URL type.

var `binaryPropertyList` → `TypeIdIdentifier` *read-only*

The binary property list type.

var `csv` → `TypeIdIdentifier` *read-only*

The comma-separated text type.

var `editableTypes` → Array of `TypeIdIdentifier` *read-only*

The list of `TypeIdIdentifiers` that can be read and written natively by documents in this application.

var `gif` → `TypeIdIdentifier` *read-only*

The GIF image type.

var `image` → `TypeIdIdentifier` *read-only*

A generic type that all image types conform to.

var jpeg → [TypeIdentifier](#) *read-only*

The JPEG image type.

var json → [TypeIdentifier](#) *read-only*

The JSON type.

var ofocus → [TypeIdentifier](#) *read-only*

The OmniFocus document file type.

var pdf → [TypeIdentifier](#) *read-only*

The PDF type.

var plainText → [TypeIdentifier](#) *read-only*

The plain text type.

var png → [TypeIdentifier](#) *read-only*

The PNG image type.

var propertyList → [TypeIdentifier](#) *read-only*

The generic property list type.

var readableTypes → Array of [TypeIdentifier](#) *read-only*

The list of `TypeIdentifiers` that can be read by documents in this application.

`var rtf → TypeIdentifier read-only`

The RTF type.

`var rtfld → TypeIdentifier read-only`

The RTFD type.

`var taskPaper → TypeIdentifier read-only`

The TaskPaper-formatted tasks pasteboard type.

`var tasks → TypeIdentifier read-only`

The pasteboard type identifier for OmniFocus tasks and projects.

`var tasksAndFolders → TypeIdentifier read-only`

The pasteboard type identifier for OmniFocus tasks, projects, and folders.

`var tiff → TypeIdentifier read-only`

The TIFF image type.

`var writableTypes → Array of TypeIdentifier read-only`

The list of `TypeIdentifier`s that can be written by documents in this application (though some documents may be exportable only in a subset of these types).

var `xmlPropertyList` → [`TypeIdentifier`](#) *read-only*

The XML property list type.

Constructors

new [`TypeIdentifier`](#)(identifier: String) → [`TypeIdentifier`](#)

Returns a new `TypeIdentifier` with the given identifier.

Instance Functions

function `conformsTo`(other: [`TypeIdentifier`](#)) → Boolean

Returns true if the instance is the same as the given argument or a more specific type. For example,

`TypeIdentifier.png.conformsTo(TypeIdentifier.image)` will be true, but `TypeIdentifier.png.conformsTo(TypeIdentifier.plainText)` will be false.

Instance Properties

var `displayName` → String *read-only*

Returns a human-readable description of the type.

var `identifier` → String *read-only*

Returns a unique string for a type identifier, suitable for archiving or encoding in scripts.

var `pathExtensions` → Array of String ***read-only***

The list of filesystem path extensions used by this type.

URL

Class Functions

function `choose`(types: Array of String) → [URL](#) or null

Deprecated: Please use `FilePicker` instead.

Allows the user to choose a file URL if possible and returns a new instance, or null otherwise.

function `chooseFolder`() → [URL](#) or null

Deprecated: Please use `FilePicker` instead.

Allows the user to choose a folder URL if possible and returns a new instance, or null otherwise.

function `fromString`(string: String, relativeToURL: [URL](#) or null) → [URL](#) or null

Parses the string as a URL if possible and returns a new instance, or null if the string is not a valid URL. If `baseURL` is not null, the result is a relative URL.

function `fromPath`(path: String, isDirectory: Boolean, relativeToURL: [URL](#) or null) → [URL](#)

Returns a new file URL with the given path and assumption of whether it is a directory.

function `tellScript`(app: String, js: String, arg: [Object](#) or null) → [URL](#) or null

Creates a URL to invoke the given JS on the given app (url scheme) appropriate for use with the call function.

function `tellFunction`(app: String, jsFunction: [Function](#), arg: [Object](#) or null) → [URL](#) or null

Creates a URL to invoke the given JS function on the given app (url scheme) appropriate for use with the call function.

Class Properties

var `currentAppScheme` → String *read-only*

Returns the URL scheme for the current app.

var `documentsDirectory` → [URL](#) *read-only*

Returns the application's Documents directory. This is in the application's sandbox, and on the Mac is **not** the user's Documents directory. This is accessible by the application without using `access()`.

Instance Functions

```
function fetch(success: Function(contents: Data),  
failure: Function(error: Error) or null)
```

Get the contents at the destination of this URL.

```
function call(success: Function, failure: Function or  
null)
```

Invoke an [x-callback-url API](#) end-point, with the callback functions being invoked when a reply is received. When a reply is received, the parameters of that URL are decoded as JSON, or left as String values if not valid JSON, and stored as properties of a result object. For a successful reply, if the result object has one property, its value is passed as the first argument to the success function. If there are zero or more than one parameters, the full object is passed as the first argument. In both cases, the success function is passed a second argument that is the full object of parameters. The failure callback is always passed the object with all the result parameters, typically `errorCode` and `errorMessage`.

```
function open()
```

Ask the system to open this URL.

```
function find(types: Array of TypeIdentifier, recurse:  
Boolean or null) → Promise of Array of URL
```

Scan a directory URL for files of the given types. If `recurse` is specified and is false, only the immediate contents of the directory will be considered. If `recurse` is not specified or is true, the full directory tree will be scanned.

function `toString()` → String

function `appendingPathComponent(component: String)` → [URL](#)

Return a new URL with the given string added at the end of the path.

function `appendingPathExtension(pathExtension: String)` → [URL](#)

Returns a new URL with the last path component having the given path extension added, including a separating “.”

function `deletingPathExtension()` → [URL](#)

Returns a new URL with the path extension (if any) of the last path component removed.

function `deletingLastPathComponent()` → [URL](#)

Returns a new URL with the last path component removed.

Instance Properties

var `absoluteString` → String *read-only*

Returns the absolute string for the URL.

var `absoluteURL` → [URL](#) *read-only*

Returns the absolute URL.

var `baseUrl` → `URL` **or null** *read-only*

Returns the base URL if this URL is relative, or `null` if it is absolute.

var `fragment` → `String` **or null** *read-only*

Returns the fragment component of the URL, or `null`.

var `hasDirectoryPath` → `Boolean` *read-only*

Returns `true` if the URL's path represents a directory.

var `host` → `String` **or null** *read-only*

Returns the host component of the URL or `null`.

var `isFileURL` → `Boolean` *read-only*

Returns `true` if the scheme is `file:`.

var `lastPathComponent` → `String` *read-only*

Returns the last component of the URL's path or an empty string.

var `password` → `String` **or null** *read-only*

Returns the password component of the URL or `null`.

var **path** → String **or null** ***read-only***

Returns the path component of the URL or null.

var **pathComponents** → Array **of** String ***read-only***

Returns the path of the URL as an array of components.

var **pathExtension** → String ***read-only***

Returns the path extension of the last path component of the URL or the empty string.

var **port** → Number **or null** ***read-only***

Returns the port component of the URL or null.

var **query** → String **or null** ***read-only***

Returns the query component of the URL or null.

var **relativePath** → String **or null** ***read-only***

Returns the relative path of the URL, or the absolute path if this URL is not relative

var **relativeString** → String ***read-only***

Returns the relative portion of the URL if it is relative, otherwise this returns the absolute string.

var `scheme` → String **or null** *read-only*

Returns the scheme of the URL.

var `string` → String *read-only*

String `absoluteString` representation of this URL.

var `user` → String **or null** *read-only*

Returns the user component of the URL or `null`.

URL.Access

A `URL.Access` holds the temporary access given by the `URL.Bookmark.access()` function. These should not be stored longer than needed.

Instance Properties

var `url` → URL *read-only*

The URL being accessed.

URL.Bookmark

A `URL.Bookmark` records the permission to access a given URL and will restore that permission, as well as a possibly renamed file, at a later point.

Class Functions

function `fromURL(url: URL) → URL.Bookmark`

Creates a `URL.Bookmark` from an existing `URL`, which should have been returned by `FilePicker`. This can then be stored in a `Credentials` object to persistently record the permission to access this `URL`.

Instance Functions

function `access() → Promise of URL.Access`

Attempts to resolve the instance into a `URL` and grant access to it through the returned `Promise`. Access to the `URL` will only last as long as the `URL.Access` object exists (which should not be stored longer than necessary).

`URL.Components`

`URL.Components` allows for correct generation and interpretation of `URL` instances, dealing with the specific different quoting rules for each specific part of the `URL`.

Class Functions

function `fromString(string: String) → URL.Components or null`

Parses the string into `URL.Components`. If the string is not a valid `URL`, `null` is returned.

function `fromURL`(url: [URL](#), resolvingAgainstBaseURL: Boolean) → [URL.Components](#) or null

Parses the string representation of the URL. If the URL's string is malformed, null is returned. If resolve is true and the given url is relative, its base URL's components are considered.

Constructors

new [URL.Components](#)() → [URL.Components](#)

Returns a new `URL.Components`.

Instance Functions

function `urlRelativeTo`(base: [URL](#) or null) → [URL](#) or null

Returns a URL relative to the base URL and the components or null.

Instance Properties

var `fragment` → String or null

var `host` → String or null

var `password` → String or null

var `path` → String

var `port` → Number or null

var `query` → String or null

var `queryItems` → Array of [URL.QueryItem](#) or null

The query of the URL.Components as individual components.

var `scheme` → String or null

var `url` → [URL](#) or null *read-only*

Returns a URL for the components or null.

var `user` → String or null

URL.FetchRequest

Class Functions

function `fromString`(string: String) → [URL.FetchRequest](#) or null

Parses the string as a URL if possible and returns a new instance, or null otherwise.

Constructors

new `URL.FetchRequest`() → [URL.FetchRequest](#)

Creates a new instance.

Instance Functions

function `fetch()` → [Promise](#) of [URL.FetchResponse](#)

Perform the request, returning a Promise. On success, the promise will resolve to a `URL.FetchResponse`.

Instance Properties

var `allowsConstrainedNetworkAccess` → Boolean

Whether connections may use the network when the user has specified Low Data Mode.

var `allowsExpensiveNetworkAccess` → Boolean

Whether connections may use a network interface that the system considers expensive.

var `bodyData` → [Data](#) or null

The body of the request, typically used in an HTTP POST or PUT request. This API is suitable for uploading binary data, or for text which needs to be encoded in a form other than UTF-8. If UTF-8 text is suitable, `bodyString` is likely to be a better choice.

var `bodyString` → String or null

The body of the request, typically used in an HTTP POST or PUT request. The provided string will be transmitted using the UTF-8 encoding.

var `cache` → String or null

The cache policy for the request: default, no-store, reload, no-cache, force-cache, or only-if-cached.

var `headers` → Object

Custom HTTP headers to be sent with this request.

var `httpShouldHandleCookies` → Boolean

Whether to automatically handle cookies.

var `httpShouldUsePipelining` → Boolean

Whether to transmit data before receiving a response from an earlier request.

var `method` → String or null

The HTTP request method of the request: GET, POST, PUT, DELETE, etc.

var `url` → [URL](#) or null

The URL for this fetch request. Much of the additional functionality provided by the fetch request API will only work with http and https URLs. (For example, the method and cache and headers don't have any effect in the context of a file or omnifocus URL.)

URL.FetchResponse

`URL.FetchResponse` represents the response from fetching a URL resource, providing additional information about the response such as the HTTP status code and headers along with the actual data for that response. This is a read-only object returned by performing a `URL.FetchRequest`; see that class for more details on actually performing the request.

Instance Properties

`var bodyData → Data or null read-only`

Returns the raw HTTP body data from this response.

`var bodyString → String or null read-only`

This is a convenience wrapper which interprets the `bodyData` of this response as UTF-8 text. (Note: the current implementation assumes the text is encoded using UTF-8, but ideally it would honor the text encoding as reported by `textEncodingName`.)

`var headers → Object read-only`

Returns the HTTP header fields for this response.

`var mimeType → String or null read-only`

Returns the HTTP MIME type for this response (e.g. `text/plain`, `application/json`, etc.).

`var statusCode → Number read-only`

Returns the HTTP status code for this response (e.g. 200, 404, etc.).

var `textEncodingName` → String **or null** *read-only*

Returns the reported text encoding for this response. This name will be the actual string reported by the origin source, or `null` if no encoding was specified.

var `url` → [URL](#) **or null** *read-only*

Returns the URL for this response.

URL.QueryItem

Constructors

new [URL.QueryItem](#)(name: String, value: String **or null**)
→ [URL.QueryItem](#)

Returns a new [URL.QueryItem](#) with the given name and value.

Instance Properties

var `name` → String *read-only*

var `value` → String **or null** *read-only*

UnderlineAffinity

Class Properties

var **ByWord** → [UnderlineAffinity](#) *read-only*

Underline only the words, but not the space between them.

var **None** → [UnderlineAffinity](#) *read-only*

Underline the entire range.

var **all** → Array of [UnderlineAffinity](#) *read-only*

UnderlinePattern

Class Properties

var **Dash** → [UnderlinePattern](#) *read-only*

Dashed line.

var **DashDot** → [UnderlinePattern](#) *read-only*

Alternating dashes and dots.

var **DashDotDot** → [UnderlinePattern](#) *read-only*

Alternating dashes and pairs of dots.

var **Dot** → [UnderlinePattern](#) *read-only*

Dotted line.

var **Solid** → [UnderlinePattern](#) *read-only*

A continuous line.

var **all** → Array of [UnderlinePattern](#) *read-only*

UnderlineStyle

Class Properties

var **Double** → [UnderlineStyle](#) *read-only*

Two lines.

var **None** → [UnderlineStyle](#) *read-only*

No underline.

var **Single** → [UnderlineStyle](#) *read-only*

A single line.

var **Thick** → [UnderlineStyle](#) *read-only*

A single thick line.

var **all** → Array of [UnderlineStyle](#) *read-only*

Version

Constructors

new `Version(versionString: String)` → `Version`

Parses a string representation of a `Version` and returns an instance, or throws an error if the string isn't a valid version.

Instance Functions

function `equals(version: Version)` → `Boolean`

Returns true if the receiving `Version` is equal to the argument `Version`.

function `atLeast(version: Version)` → `Boolean`

Returns true if the receiving `Version` is at the same as or newer than the argument `Version`.

function `isAfter(version: Version)` → `Boolean`

Returns true if the receiving `Version` is strictly after the argument `Version`.

function `isBefore(version: Version)` → `Boolean`

Returns true if the receiving `Version` is strictly before the argument `Version`.

Instance Properties

var `versionString` → `String` *read-only*

Returns as an opaque string representation of the `Version`, suitable for display or logging. This should never be used in comparisons of any sort.

Window

Instance Functions

function `close()`

DocumentWindow : Window

Instance Functions

function `selectObjects(objects: Array of DatabaseObject)`

Clears the current selection and then selects the given objects, if present in the current perspective of this window. On iOS, if `objects` contains more than one object, this will put the outline view into edit mode to accomodate multiple selection.

function `forecastDayForDate(date: Date) → ForecastDay`

Returns a `ForecastDay` object that encompasses `date`. This will throw an error if `Forecast` is not the current perspective in this window.

function `selectForecastDays(days: Array of ForecastDay)`

Selects the days in the Forecast picker represented by days. On iOS, only the first day is selected, and the rest are ignored. This will throw an error if Forecast is not the current perspective in this window.

Instance Properties

var `content` → [ContentTree](#) or null *read-only*

The tree of nodes representing the content area of the window.

var `focus` → [SectionArray](#) or null

The Folders and Projects that the window is focusing on, limiting the sidebar to show only these items. Focus is only supported on macOS- getting or setting this on iOS results in an error.

var `inspectorVisible` → Boolean

Whether the inspector is currently visible in the window. On iOS, showing this pane may implicitly hide other panes and may be only transiently visible, depending on the available space.

var `isTab` → Boolean *read-only*

Whether or not this window is a tab. This is not available on iOS.

var `perspective` → [Perspective.BuiltIn](#) or [Perspective.Custom](#) or null

var `selection` → [Selection](#) *read-only*

The current selection in the window.

var `sidebar` → [SidebarTree](#) or null *read-only*

The tree of nodes representing the sidebar of the window.

var `sidebarVisible` → Boolean

Whether the sidebar is currently visible in the window. On iOS, showing this pane may implicitly hide other panes and may be only transiently visible, depending on the available space.

var `tabGroupWindows` → Array of [DocumentWindow](#) *read-only*

The array of sibling Window objects that are in tabs alongside this Window. If `isTab` is false, then this will return an array that solely contains this Window. This is not available on iOS.

XML

XML.Document

Class Functions

function `fromData`(data: [Data](#), whitespaceBehavior: [XML.WhitespaceBehavior](#) or null) → [XML.Document](#)

Parse the given data as an XML document.

Constructors

`new XML.Document(rootElement: String or XML.Element, configuration: XML.Document.Configuration or null) → XML.Document`

Returns a new XML.Document with the given root element and configuration.

Instance Functions

`function xmlData() → Data`

Encodes the document as XML.

`function addElement(name: String, function: Function() or null)`

Appends a new element with the given name. If a function is passed, it is pushed it on the current element stack, the supplied function is called, and then the element is popped off the stack.

`function appendString(string: String)`

Appends the given string as a child of topElement.

`function setAttribute(attribute: String, value: String or null)`

Sets the specified attribute on topElement.

Instance Properties

var `dtdPublicID` → String **or null** *read-only*

var `dtdSystemID` → [URL](#) **or null** *read-only*

var `rootElement` → [XML.Element](#) *read-only*

var `schemaID` → [URL](#) **or null** *read-only*

var `schemaNamespace` → String **or null** *read-only*

var `stringEncoding` → [StringEncoding](#) *read-only*

var `topElement` → [XML.Element](#) *read-only*

Returns the element at the top of the current element stack. Initially this is the root element, but when `addElement()` is called, it is temporarily updated to the new element (possibly recursively).

var `whitespaceBehavior` → [XML.WhitespaceBehavior](#) *read-only*

XML.Document.Configuration

Constructors

new `XML.Document.Configuration()` →
[XML.Document.Configuration](#)

Returns a new `XML.Document.Configuration` with default settings.

Instance Properties

var `dtdPublicID` → String **or** null

var `dtdSystemID` → [URL](#) **or** null

var `schemaID` → [URL](#) **or** null

var `schemaNamespace` → String **or** null

var `stringEncoding` → [StringEncoding](#)

var `whitespaceBehavior` → [XML.WhitespaceBehavior](#) **or** null

XML.Element

Constructors

new `XML.Element`(name: String) → [XML.Element](#)

Returns a new `XML.Element` with the given name.

Instance Functions

function `childAtIndex`(childIndex: Number) → String **or** [XML.Element](#) **or** null

Returns the child at the given index, or nil if the index is past the last child.

function `insertChild`(child: String **or** [XML.Element](#),
childIndex: Number)

Inserts the new child at the specified index. If the index is past the end of the current children, it is appended instead.

```
function appendChild(child: String or XML.Element)
```

Adds the new item to the end of the children.

```
function removeChildAtIndex(childIndex: Number)
```

Removes the child at the given index. If the index is past the end of the current children, no removal occurs.

```
function removeAllChildren()
```

Removes any existing children.

```
function firstChildNamed(name: String) → XML.Element or  
null
```

Returns the first child element with the given name, or `null` if there is no such child.

```
function firstChildAtPath(path: String) → XML.Element or  
null
```

Given a path which is a string separated by `"/"`, returns the first element at that path.

```
function firstChildWithAttribute(attribute: String,  
value: String) → XML.Element or null
```

Returns the first child with an attribute set to the given value.

```
function attributeNamed(name: String) → String or null
```

Returns the value of the the given attribute or `null` if no value has been assigned.

```
function setAttribute(name: String, value: String or null)
```

Sets the value for the specified attribute. If the element already had a value for this attribute, it is replaced in place. If there previously was no value for this attribute, the attribute is appended to `attributeNames`. If the new value is `null`, the attribute is removed.

```
function apply(function: Function(node: String or XML.Element) → \_omnajs\_AnonymousProxy or null) → \_omnajs\_AnonymousProxy or null
```

Calls the supplied function for each child element or string in the receiver (including the receiver), passing that child as the single argument. The supplied function can optionally return a `ApplyResult` to skip enumeration of some elements.

Instance Properties

```
var attributeCount → Number read-only
```

Returns the number of attributes assigned to this element.

```
var attributeNames → Array of String read-only
```

Returns the names of the attributes in the order they were added to the element.

var `children` → Array of String or [XML.Element](#)

The current child strings and elements.

var `childrenCount` → Number *read-only*

Returns the current count of child strings and elements.

var `lastChild` → String or [XML.Element](#) or null *read-only*

Returns the last child of the element, or null if there are no children.

var `name` → String *read-only*

Returns the name of the element.

var `stringContents` → String *read-only*

Gathers all the immediate and descendent string children and returns them concatenated them as single string.

XML.WhiteSpaceBehavior

Constructors

new [XML.WhiteSpaceBehavior](#)(defaultBehavior: [XML.WhiteSpaceBehavior.Type](#)) → [XML.WhiteSpaceBehavior](#)

Returns a new `XML.WhitespaceBehavior` with the given default type.

Instance Functions

```
function setBehaviorForElementName(behavior:
XML.WhitespaceBehavior.Type, elementName: String)
```

```
function behaviorForElementName(elementName: String) →
XML.WhitespaceBehavior.Type
```

Instance Properties

```
var defaultBehavior → XML.WhitespaceBehavior.Type read-only
```

XML.WhitespaceBehavior.Type

Class Properties

```
var Auto → XML.WhitespaceBehavior.Type read-only
```

Inherit the behavior from the enclosing element

```
var Ignore → XML.WhitespaceBehavior.Type read-only
```

Ignore whitespace

```
var Preserve → XML.WhitespaceBehavior.Type read-only
```

Preserve whitespace

var `all` → Array of `XML.WhitespaceBehavior.Type` ***read-only***